

程式人《十分鐘看不完系列》



如何設計電腦

還有讓電腦變快的那些方法

陳鍾誠 2018 年 1 月 1 日

電腦

- 其實就是計算機 ...

在計算機的世界裡

- 所有資料都是 0 與 1 組成的

所有的計算

- 都是透過 0 與 1 的運算所形成的

0 與 1 的運算

- 都可以用下列三種邏輯閘完成

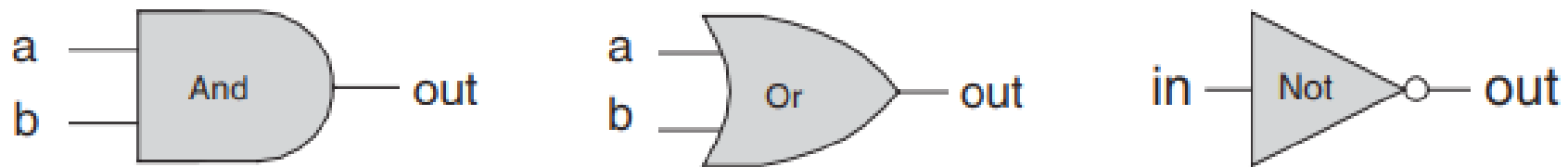
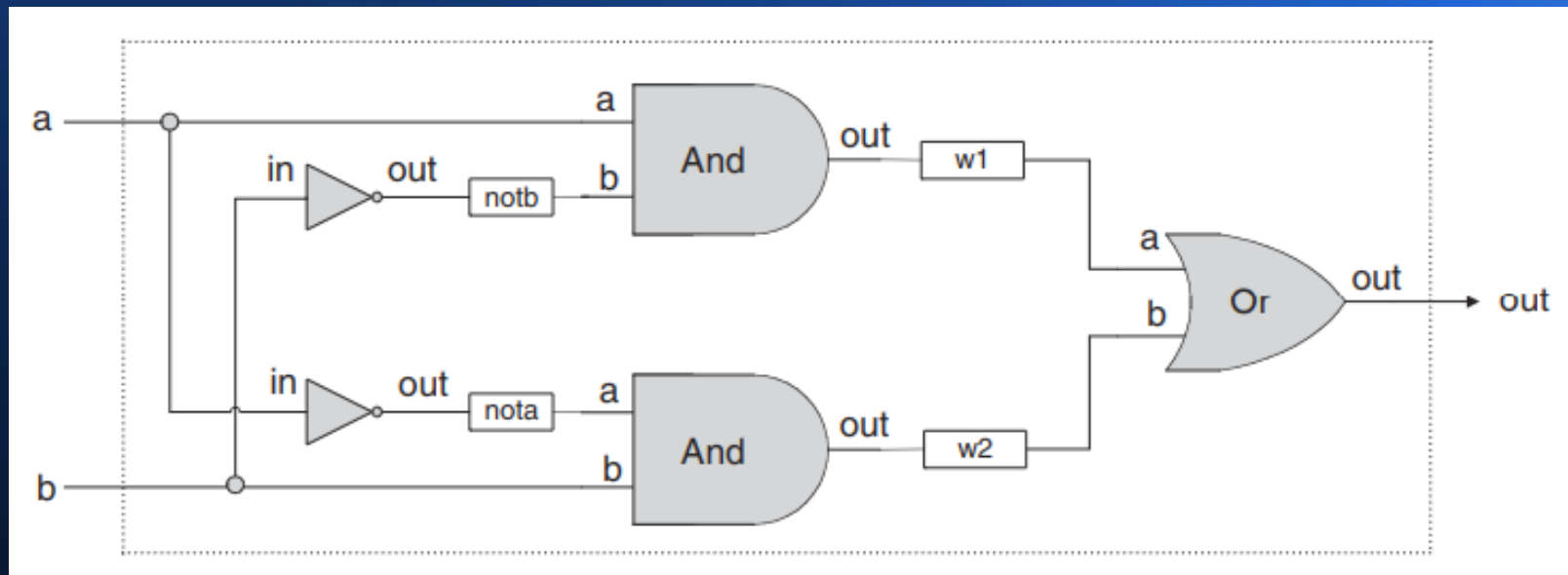


Figure 1.3 Standard symbolic notation of some elementary logic gates.

這些邏輯閘的組合

- 可以形成更複雜的電路



xor

我們可以寫《硬體描述語言》

- 設計這些電路，然後用測試程式驗證電路

<i>HDL program</i> (Xor.hdl)	<i>Test script</i> (Xor.tst)	<i>Output file</i> (Xor.out)															
<pre>/* Xor (exclusive or) gate: If a<>b out=1 else out=0. */ CHIP Xor { IN a, b; OUT out; PARTS: Not(in=a, out=nota); Not(in=b, out=notb); And(a=a, b=notb, out=w1); And(a=nota, b=b, out=w2); Or(a=w1, b=w2, out=out); }</pre>	<pre>load Xor.hdl, output-list a, b, out; set a 0, set b 0, eval, output; set a 0, set b 1, eval, output; set a 1, set b 0, eval, output; set a 1, set b 1, eval, output;</pre>	<table><thead><tr><th>a</th><th>b</th><th>out</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></tbody></table>	a	b	out	0	0	0	0	1	1	1	0	1	1	1	0
a	b	out															
0	0	0															
0	1	1															
1	0	1															
1	1	0															

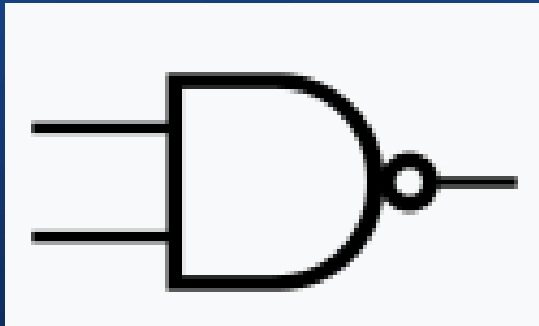
Figure 1.6 HDL implementation of a Xor gate.

你可以在 nand2tetris 這門課

<http://nand2tetris.org/>

- 學到從頭設計一台電腦的方法
- 課程從硬體到軟體的設計都很完整

nand2teris 課程從 NAND 開始



$$Y = \overline{A B}$$

<i>a</i>	<i>b</i>	Nand(<i>a</i> , <i>b</i>)
0	0	1
0	1	1
1	0	1
1	1	0

Chip name: Nand

Inputs: a, b

Outputs: out

Function: If a=b=1 then out=0 else out=1

Comment: This gate is considered primitive and thus there is no need to implement it.

只要給夠多的 nand 閘

- 就能將一台電腦從《多工器、全加器、加法器、ALU、暫存器、記憶體到CPU》全部設計完成。

這些 nand2tetris 的習題

- 我都做過一遍了
- 也在計算機結構的課程
讓學生從頭到尾做一遍 ...

nand2tetris 的解答我放在這裡

- <https://github.com/ccckmit/nand2tetris/tree/master/01>

<https://github.com/ccckmit/nand2tetris/tree/master/02>

<https://github.com/ccckmit/nand2tetris/tree/master/03>

<https://github.com/ccckmit/nand2tetris/tree/master/04>

<https://github.com/ccckmit/nand2tetris/tree/master/05>

以上

- 是 nand2tetris 課程一到五章的習題解答

也就是硬體設計的全部章節

(後面的 6-12 章是軟體部分)

其中比較重要的一些元件

- 是《多工器、全加器、加法器、暫存器、記憶體、CPU》等等

我們在此將程式碼列出來

```
/**
 * Multiplexor:
 * out = a if sel == 0
 *      b otherwise
 */

CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
    Not(in=sel, out=nsel);
    And(a=a, b=nsel, out=o1);
    And(a=b, b=sel, out=o2);
    Or(a=o1, b=o2, out=out);
}
```

多工器 MUX

```
/**
 * Demultiplexor:
 * {a, b} = {in, 0} if sel == 0
 *         {0, in} if sel == 1
 */

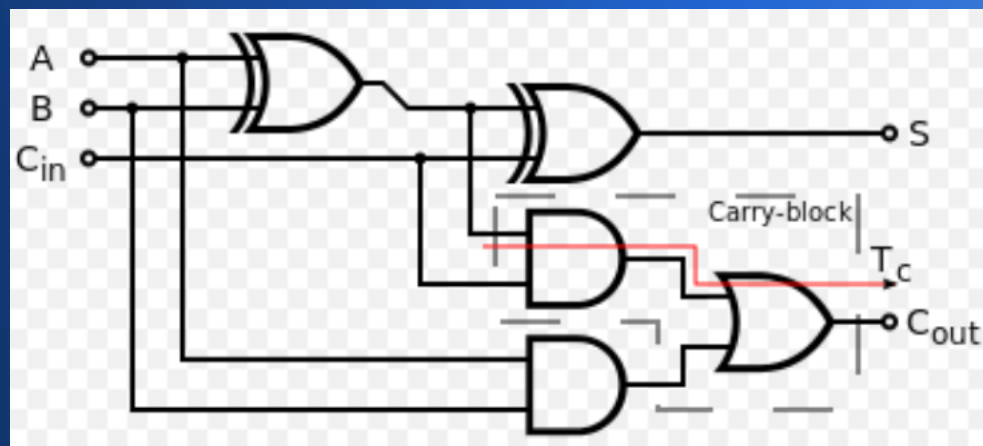
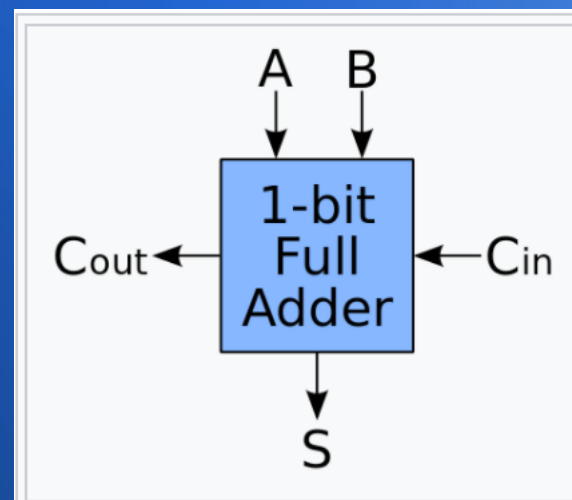
CHIP DMux {
    IN in, sel;
    OUT a, b;

    PARTS:
    Not(in=sel, out=nsel);
    And(a=nsel, b=in, out=a);
    And(a=sel, b=in, out=b);
}
```

解多工器 DMUX

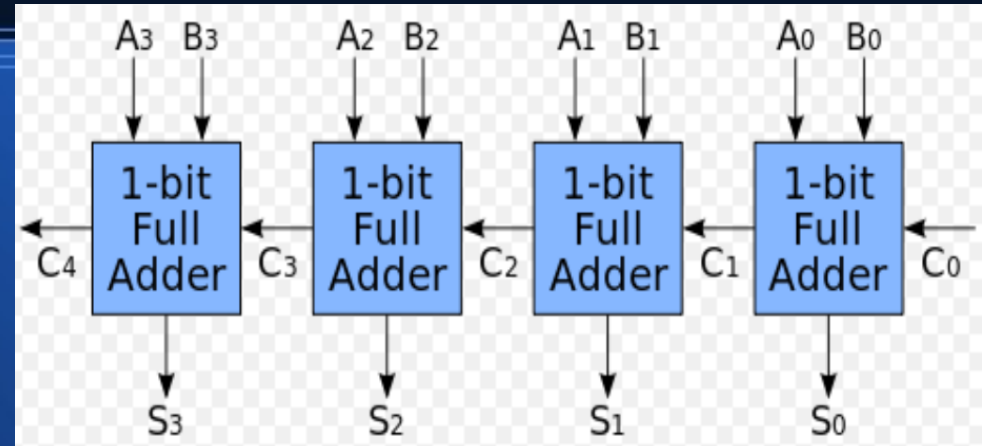
全加器

```
CHIP FullAdder {  
    IN a, b, c; // 1-bit inputs  
    OUT sum,    // Right bit of a + b + c  
        carry; // Left bit of a + b + c  
  
    PARTS:  
        Xor(a=a, b=b, out=s1);  
        Xor(a=s1, b=c, out=sum);  
        And(a=a, b=b, out=c1);  
        And(a=s1, b=c, out=c2);  
        Xor(a=c2, b=c1, out=carry);  
}
```



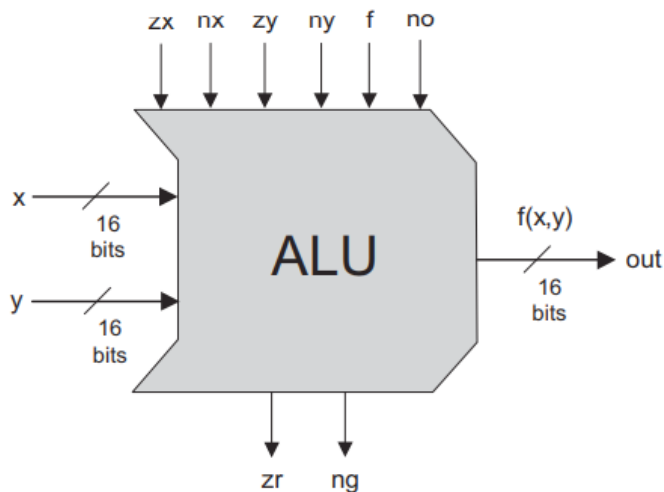
16 位元加法器

```
CHIP Add16 {  
  IN a[16], b[16];  
  OUT out[16];  
  
  PARTS:  
  FullAdder(a=a[0], b=b[0], c=false, sum=out[0], carry=c0);  
  FullAdder(a=a[1], b=b[1], c=c0, sum=out[1], carry=c1);  
  FullAdder(a=a[2], b=b[2], c=c1, sum=out[2], carry=c2);  
  FullAdder(a=a[3], b=b[3], c=c2, sum=out[3], carry=c3);  
  FullAdder(a=a[4], b=b[4], c=c3, sum=out[4], carry=c4);  
  FullAdder(a=a[5], b=b[5], c=c4, sum=out[5], carry=c5);  
  FullAdder(a=a[6], b=b[6], c=c5, sum=out[6], carry=c6);  
  FullAdder(a=a[7], b=b[7], c=c6, sum=out[7], carry=c7);  
  FullAdder(a=a[8], b=b[8], c=c7, sum=out[8], carry=c8);  
  FullAdder(a=a[9], b=b[9], c=c8, sum=out[9], carry=c9);  
  FullAdder(a=a[10], b=b[10], c=c9, sum=out[10], carry=c10);  
  FullAdder(a=a[11], b=b[11], c=c10, sum=out[11], carry=c11);  
  FullAdder(a=a[12], b=b[12], c=c11, sum=out[12], carry=c12);  
  FullAdder(a=a[13], b=b[13], c=c12, sum=out[13], carry=c13);  
  FullAdder(a=a[14], b=b[14], c=c13, sum=out[14], carry=c14);  
  FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);  
}
```



上圖是四位元加法器的圖示
繼續一路串接成 16 個的話
就會是左邊的 16 位元加法器了！

算術邏輯單元 ALU



These bits instruct
how to preset
the x input

These bits instruct
how to preset
the y input

This bit selects
between
+ / And

This bit inst.
how to
postset out

Resulting
ALU
output

zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y

ALU 的圖示

ALU 的運算表格

Figure 2.6 The ALU truth table. Taken together, the binary operations coded by the first six columns effect the function listed in the right column (we use the symbols !, &, and | to represent the operators Not, And, and Or, respectively, performed bit-wise). The complete ALU truth table consists of sixty-four rows, of which only the eighteen presented here are of interest.

算術邏輯單元 ALU 的實作程式

```
// Implementation: the ALU logic manipulates the x and y inputs
// and operates on the resulting values, as follows:
// if (zx == 1) set x = 0      // 16-bit constant
// if (nx == 1) set x = !x     // bitwise not
// if (zy == 1) set y = 0      // 16-bit constant
// if (ny == 1) set y = !y     // bitwise not
// if (f == 1) set out = x + y // integer 2's complement addition
// if (f == 0) set out = x & y // bitwise and
// if (no == 1) set out = !out  // bitwise not
// if (out == 0) set zr = 1
// if (out < 0) set ng = 1
```

```
CHIP ALU {
    IN
        x[16], y[16], // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f, // compute out = x + y (if 1) or x & y (if 0)
        no; // negate the out output?

    OUT
        out[16], zr, ng;
```

PARTS:

```
Mux16(a=x, b=false, sel=zx, out=x1); // if (zx == 1) set x = 0
Not16(in=x1,out=notx1);
Mux16(a=x1, b=notx1, sel=nx, out=x2); // if (nx == 1) set x = !x

Mux16(a=y, b=false, sel=zy, out=y1); // if (zy == 1) set y = 0
Not16(in=y1,out=noty1);
Mux16(a=y1, b=noty1, sel=ny, out=y2); // if (ny == 1) set y = !y

Add16(a=x2, b=y2, out=addxy); // addxy = x + y
And16(a=x2, b=y2, out=andxy); // andxy = x & y

// else set out = x & y
Mux16(a=andxy, b=addxy, sel=f, out=o1); // if (f == 1) set out = x + y
Not16(in=o1, out=noto1);

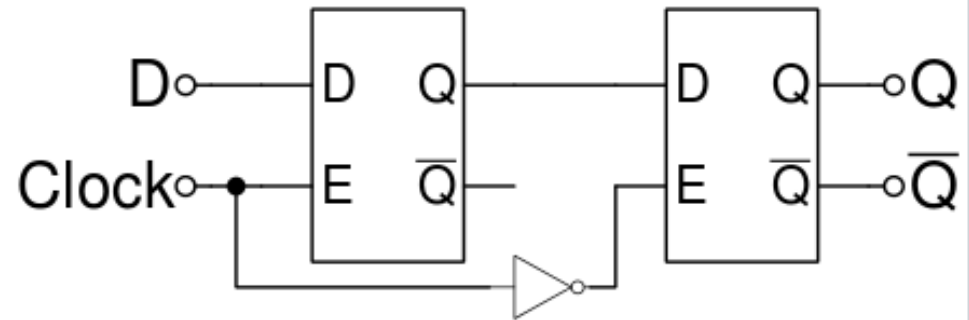
Mux16(a=o1, b=noto1, sel=no, out=o2); // if (no == 1) set out = !out

// o2 就是 out, 但必須中間節點才能再次當作輸入, 所以先用 o2 =
And16(a=o2, b=o2, out[0..7]=outLow, out[8..15]=outHigh);
Or8Way(in=outLow, out=orLow); // orLow = Or(out[0..7]);
Or8Way(in=outHigh, out=orHigh); // orHigh = Or(out[8..15]);
Or(a=orLow, b=orHigh, out=notzr); // nzr = Or(out[0..15]);
Not(in=notzr, out=zr); // zr = !nzr
And16(a=o2, b=o2, out[15]=ng); // ng = out[15]
And16(a=o2, b=o2, out=out);
```

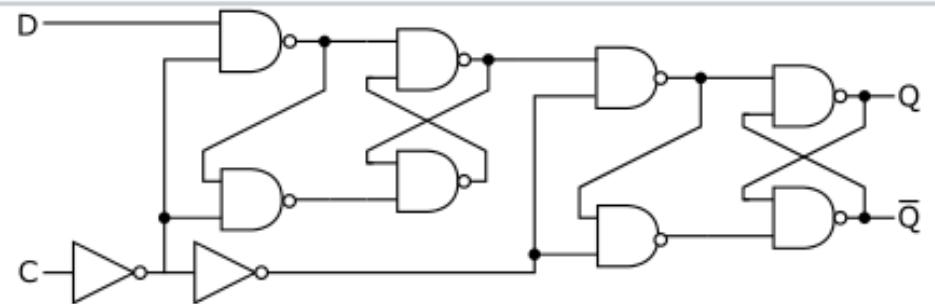
}

接著老師直接給了 DFF 邊緣觸發 D 型正反器

- 這個其實是可以自己設計的
- 一個《邊緣觸發 D 正反器》，可以用《兩個 D 正反器》組成



A master-slave D flip-flop. It responds on the falling edge of the *enable* input (usually a clock)

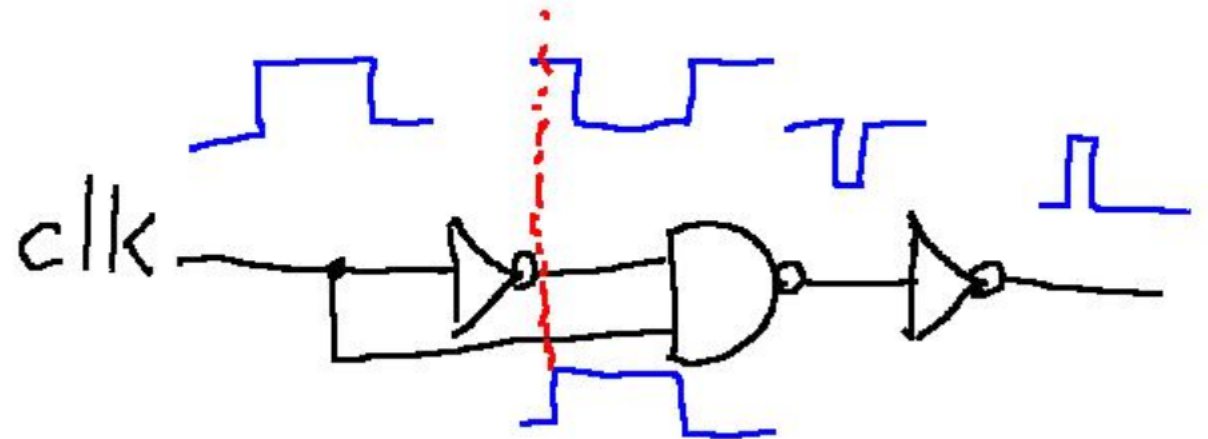


An implementation of a master-slave D flip-flop that is triggered on the rising edge of the clock

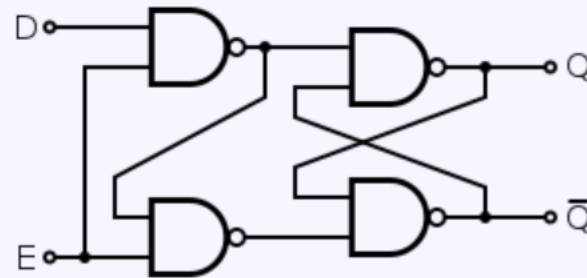
DFF 邊緣觸發正反器

也可以用《一個 D 正反器》加上《脈衝偵測電路》組成這樣會更簡單一點

脈衝偵測電路

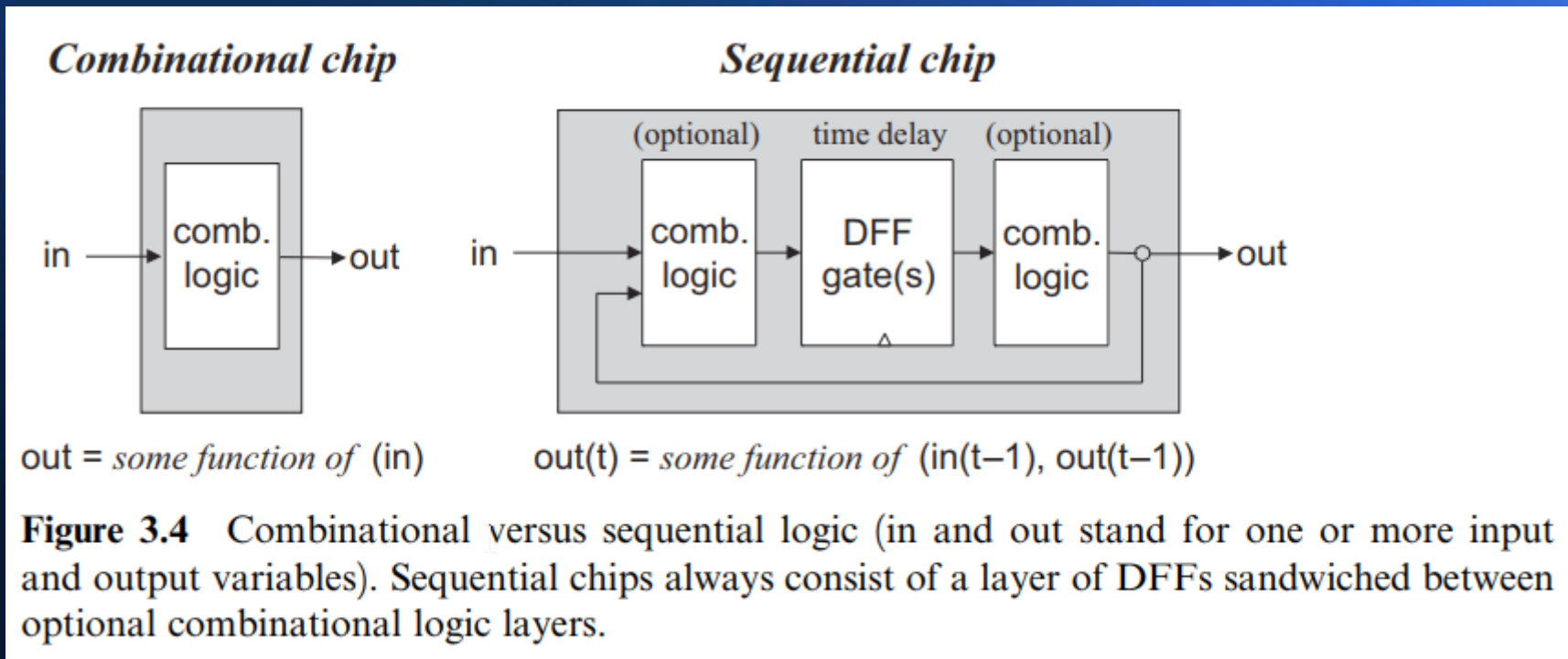


D 型正反器



這種 DFF 邊緣觸發元件

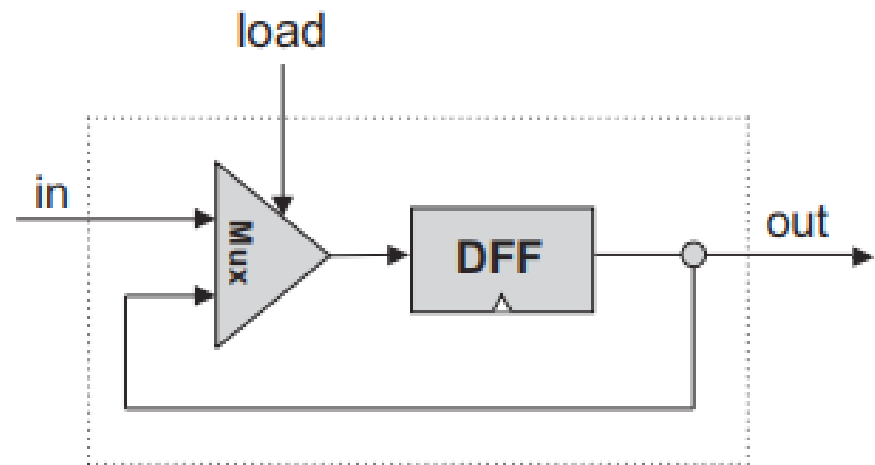
- 可以將電路分隔成兩塊，輸出會比輸入慢一拍，有記憶效果，也有類似牆壁的隔絕效果



然後我們可以用《邊緣觸發 DFF》 設計出單一位元的 bit 儲存器

```
/**  
 * 1-bit register:  
 * If load[t] == 1 then out[t+1] = in[t]  
 *           else out does not change (out[t+1] = out[t])  
 */
```

```
CHIP Bit {  
    IN in, load;  
    OUT out;  
  
    PARTS:  
    Mux(a=do, b=in, sel=load, out=mo);  
    DFF(in=mo, out=do);  
    And(a=do, b=do, out=out);  
}
```

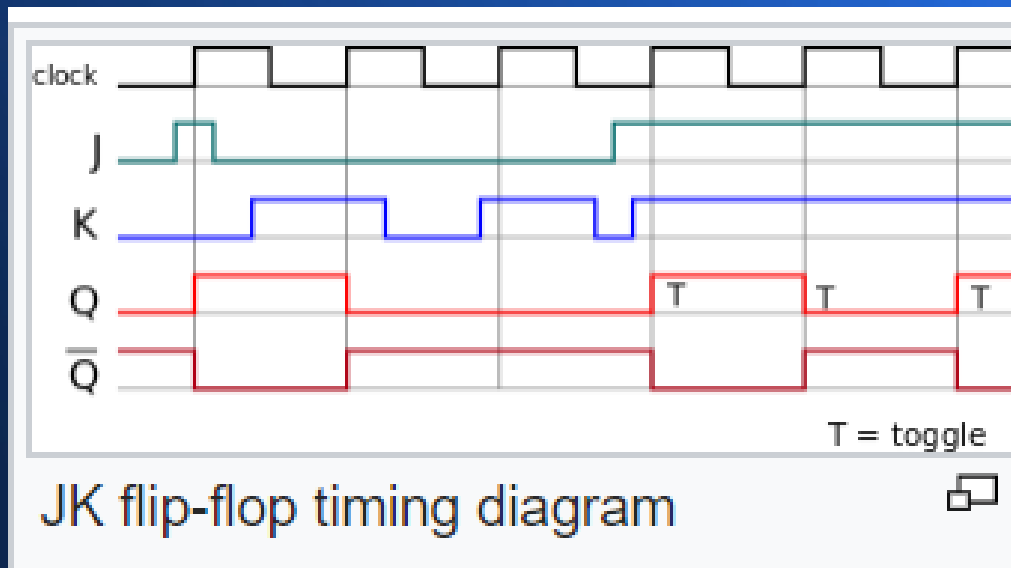
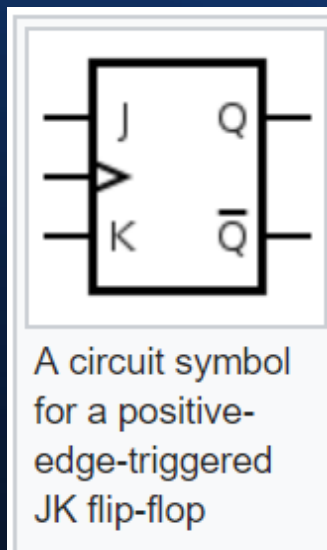


if load(t-1) then out(t) = in(t-1)
else out(t) = out(t-1)

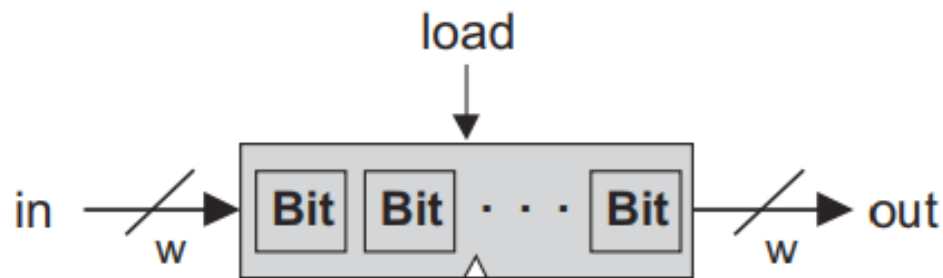
1-bit register (Bit)

邊緣觸發的元件

- 只有在時脈的正邊緣，才會發生資料改變的情況



暫存器可以由 16 個 bit 組成



if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

w -bit register

暫存器的結構

硬體描述語言的程式碼

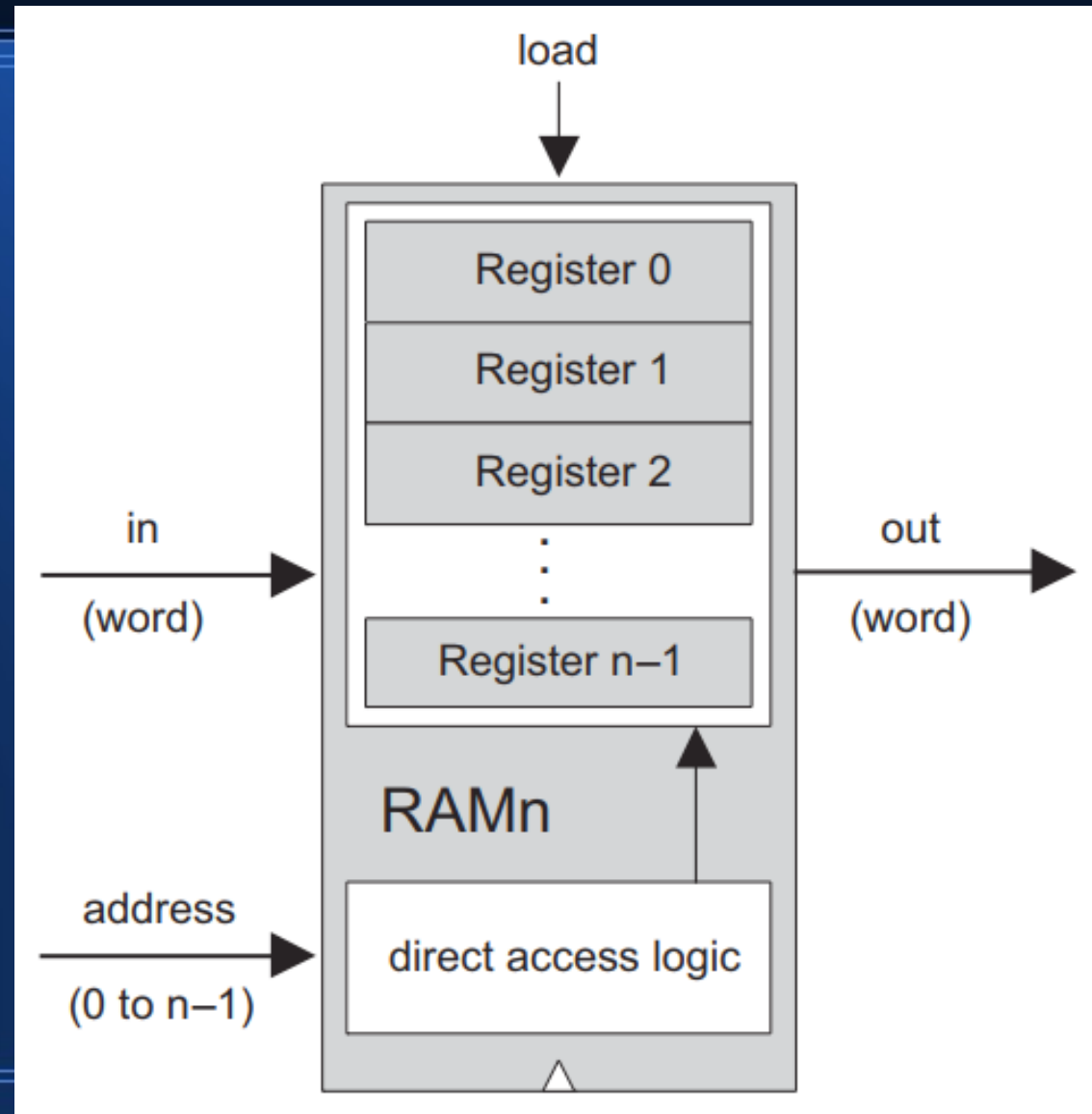
```
CHIP Register {  
    IN in[16], load;  
    OUT out[16];  
  
    PARTS:  
        Bit(in=in[15], load=load, out=out[15]);  
        Bit(in=in[14], load=load, out=out[14]);  
        Bit(in=in[13], load=load, out=out[13]);  
        Bit(in=in[12], load=load, out=out[12]);  
        Bit(in=in[11], load=load, out=out[11]);  
        Bit(in=in[10], load=load, out=out[10]);  
        Bit(in=in[9], load=load, out=out[9]);  
        Bit(in=in[8], load=load, out=out[8]);  
        Bit(in=in[7], load=load, out=out[7]);  
        Bit(in=in[6], load=load, out=out[6]);  
        Bit(in=in[5], load=load, out=out[5]);  
        Bit(in=in[4], load=load, out=out[4]);  
        Bit(in=in[3], load=load, out=out[3]);  
        Bit(in=in[2], load=load, out=out[2]);  
        Bit(in=in[1], load=load, out=out[1]);  
        Bit(in=in[0], load=load, out=out[0]);  
}
```

必須特別強調的是

- 由於暫存器是由 DFF 形成的 bit 所組成，因此也有類似牆壁的隔絕效果
- 這種隔絕效果對後面要講解的 《管線架構》很重要
- 因為暫存器將 CPU 隔開成多個區塊後，才能讓各區塊分別執行不同運算，而且不會互相干擾。

接著是記憶體

在 nand2tetris 課程的習題裏，記憶體是由一堆暫存器加上控制單元組成的



以下是由 8 個暫存器 所形成的記憶體 RAM8

```
CHIP RAM8 {  
    IN in[16], load, address[3];  
    OUT out[16];  
  
    PARTS:  
        DMux8Way(in=load, sel=address, a=L0, b=L1, c=L2, d=L3, e=L4, f=L5, g=L6, h=L7);  
  
        Register(in=in, load=L0, out=r0);  
        Register(in=in, load=L1, out=r1);  
        Register(in=in, load=L2, out=r2);  
        Register(in=in, load=L3, out=r3);  
        Register(in=in, load=L4, out=r4);  
        Register(in=in, load=L5, out=r5);  
        Register(in=in, load=L6, out=r6);  
        Register(in=in, load=L7, out=r7);  
  
        Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7, sel=address, out=out);  
}
```

然後用 8 個 RAM8 加上控制單元就可以得到 RAM64

```
CHIP RAM64 {  
    IN in[16], load, address[6];  
    OUT out[16];  
  
    PARTS:  
    DMux8Way(in=load, sel=address[3..5], a=L0, b=L1, c=L2, d=L3, e=L4, f=L5, g=L6, h=L7);  
  
    RAM8(in=in, load=L0, address=address[0..2], out=o0);  
    RAM8(in=in, load=L1, address=address[0..2], out=o1);  
    RAM8(in=in, load=L2, address=address[0..2], out=o2);  
    RAM8(in=in, load=L3, address=address[0..2], out=o3);  
    RAM8(in=in, load=L4, address=address[0..2], out=o4);  
    RAM8(in=in, load=L5, address=address[0..2], out=o5);  
    RAM8(in=in, load=L6, address=address[0..2], out=o6);  
    RAM8(in=in, load=L7, address=address[0..2], out=o7);  
  
    Mux8Way16(a=o0, b=o1, c=o2, d=o3, e=o4, f=o5, g=o6, h=o7, sel=address[3..5], out=out);  
}
```

接著一路從 RAM8, 64, 512, 4K
直到做出 16K 的記憶體，就夠我們用了

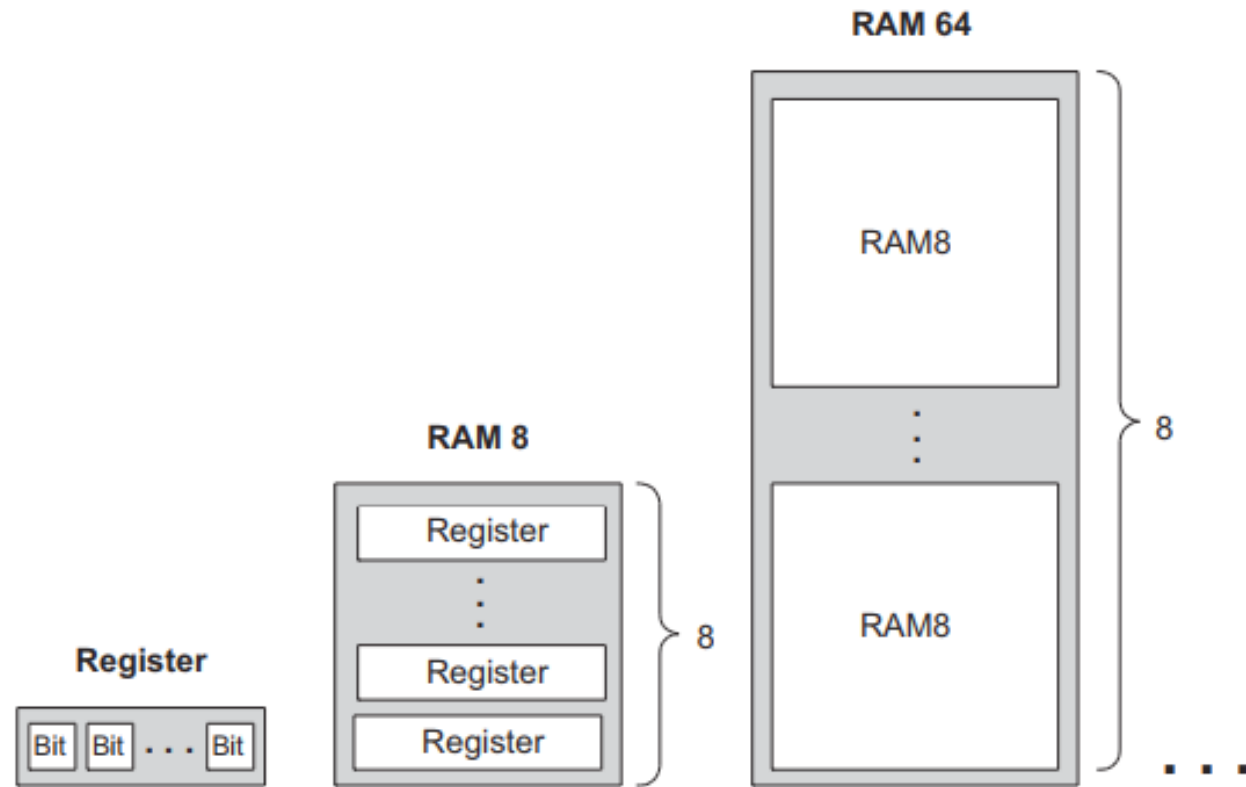


Figure 3.6 Gradual construction of memory banks by recursive ascent. A w -bit register is an array of w binary cells, an 8-register RAM is an array of eight w -bit registers, a 64-register RAM is an array of eight RAM8 chips, and so on. Only three more similar construction steps are necessary to build a 16K RAM chip.

但是這種做法的記憶體會很貴

- 通常只有《第一級快取記憶體》才會這樣子做

大部分電腦中的記憶體階層如下四層：

1) 暫存器—可能是最快的存取。在32位元處理器，每個暫存器就是32位元。x86處理器共有16個暫存器。

2) 快取（L1-L3: SRAM）

第一級快取（L1）—通常存取只需要幾個週期，通常是幾十個KB。

第二級快取（L2）—比L1約有2到10倍較高延遲性，通常是幾百個KB或更多。

第三級快取（L3）（不一定有）—比L2更高的延遲性，通常有數MB之大。

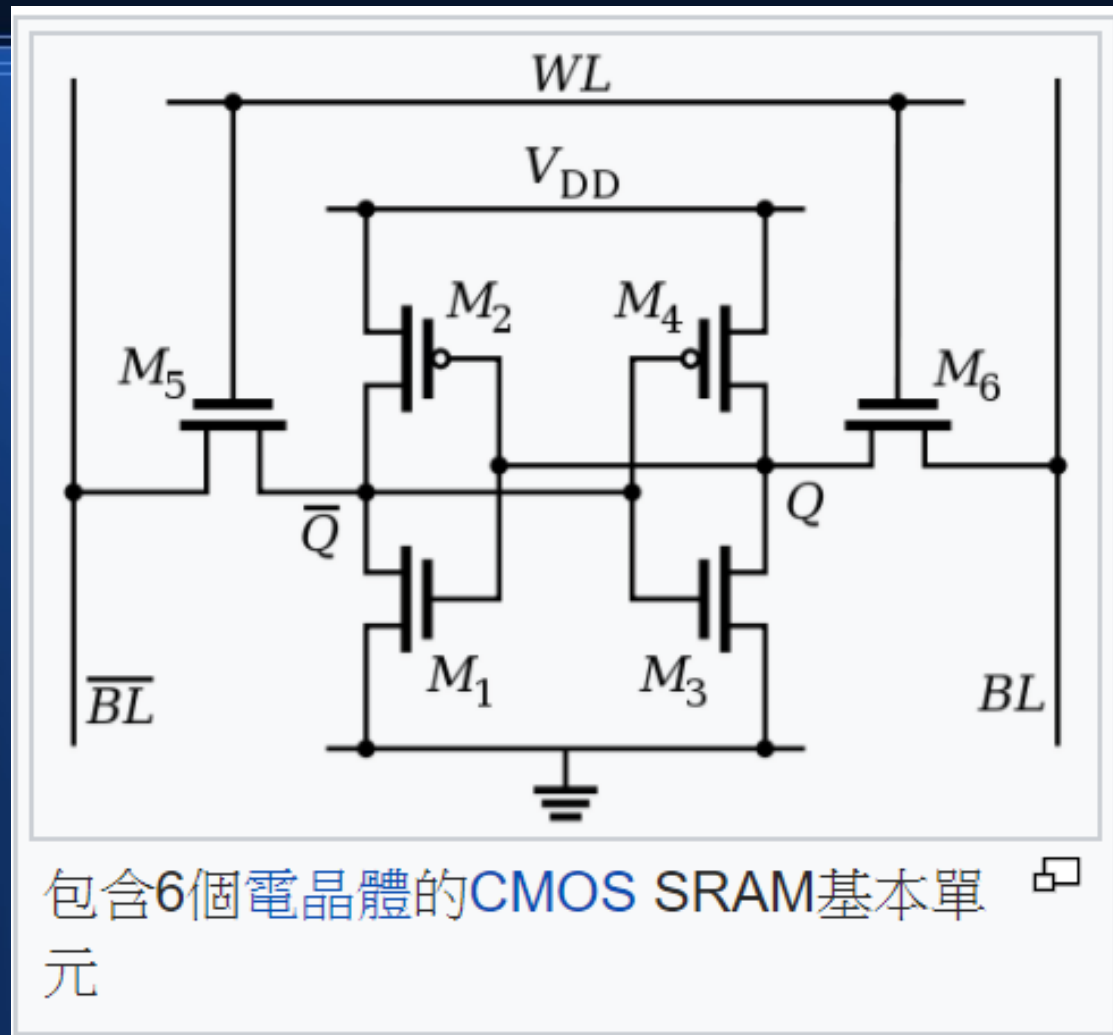
第四級快取（L4）（不普遍）—CPU外部的DRAM，但速度較主記憶體高。

3) 主記憶體（DRAM）—存取需要幾百個週期，可以大到數十GB。

4) 磁碟儲存—需要成千上百個週期，容量非常大。

而且從電子學的角度看

- 可以直接用 CMOS 組成記憶體
- 不需要用邏輯閘的想法
- 這樣電路會更精簡



下層會用其他較慢的記憶體

- 較便宜且容量較大，但是速度較慢

像是 DRAM 就是用《微小電容》所做的



動態隨機存取記憶體（**Dynamic Random Access Memory**，**DRAM**）是一種半導體記憶體，主要的作用原理是利用電容內儲存電荷的多寡來代表一個二進位位元（bit）是1還是0。由於在現實中電容會有漏電的現象，導致電位差不足而使記憶消失，因此除非電容經常周期性地充電，否則無法確保記憶長存。由於這種需要定時重新整理的特性，因此被稱為「動態」記憶體。相對來說，靜態記憶體（**SRAM**）只要存入資料後，縱使不重新整理也不會遺失記憶。

與SRAM相比，DRAM的優勢在於結構簡單——每一個位元的資料都只需一個電容跟一個電晶體來處理，相比之下在SRAM上一個位元通常需要六個電晶體。正因這緣故，DRAM擁有非常高的密度，單位體積的容量較高因此成本較低。但相反的，DRAM也有存取速度較慢，耗電量較大的缺點。

與大部分的隨機存取記憶體（RAM）一樣，由於存在DRAM中的資料會在電力切斷以後很快消失，因此它屬於一種揮發性記憶體（volatile memory）裝置。

有了以上這些

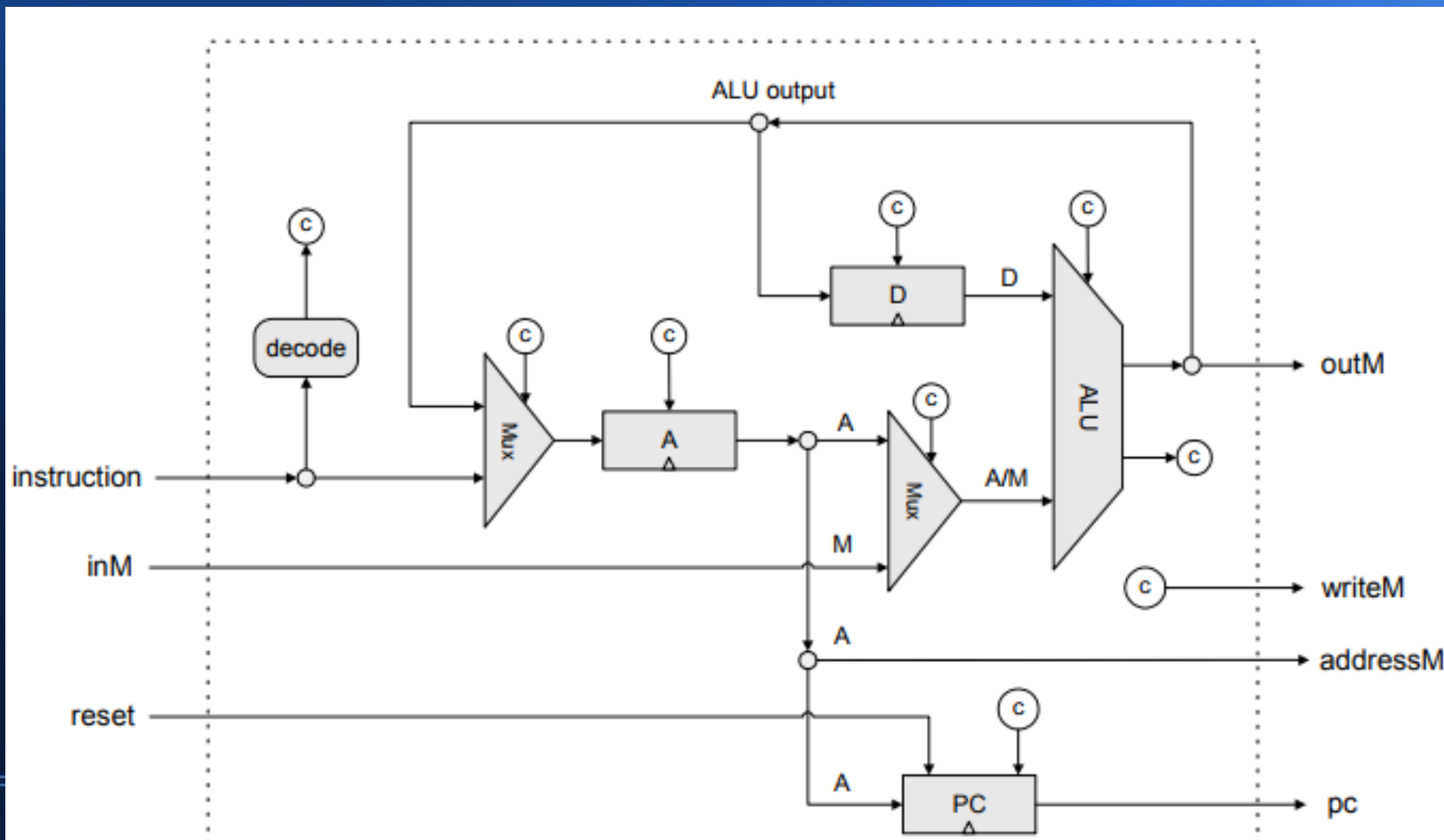
- 我們就可以建構出 CPU 了

Nand2tetris 課程

- 裡面的 CPU 稱為 HackCPU

以下是 HackCPU 的結構

(你必須自行將 c 符號的控制電路設計實作出來)

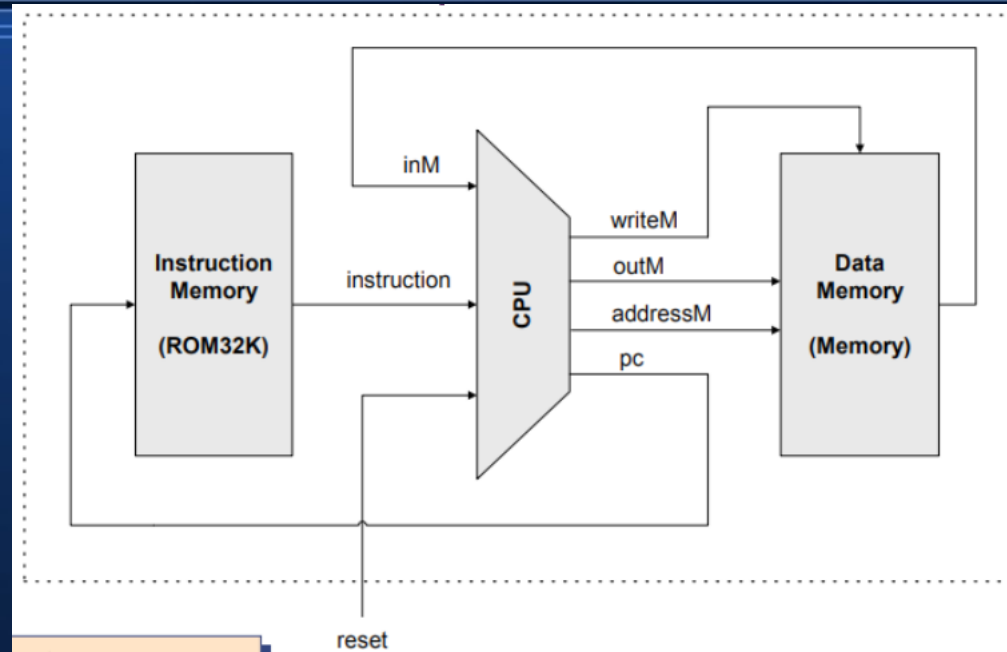


這題很有挑戰性

```
CHIP CPU {  
  
    IN  inM[16],          // M value input  (M = contents of RAM[A])  
        instruction[16], // Instruction for execution  
        reset;           // Signals whether to re-start the current  
                          // program (reset==1) or continue executing  
                          // the current program (reset==0).  
  
    OUT outM[16],         // M value output  
        writeM,          // Write to M?  
        addressM[15],    // Address in data memory (of M)  
        pc[15];          // address of next instruction  
  
    PARTS:  
        // A register  
        Not(in=instruction[15], out=isA);          // isA = not I15  
        Not(in=isA, out=isC);                     // isC = I15  
        And(a=isC, b=instruction[5], out=AluToA); // AluToA = isC & d1  
        Or(a=isA, b=AluToA, out=Aload);  
  
        Mux16(a=instruction, b=ALUout, sel=isC, out=Ain);  
        ARegister(in=Ain, load=Aload, out=Aout);  
  
        // D register  
        And(a=isC, b=instruction[4], out=Dload); // Dload = isC & d2  
        DRegister(in=ALUout, load=Dload, out=Dout);  
  
        // ALU  
        Mux16(a=Aout, b=inM, sel=instruction[12], out=AorM); // cAorM = I[12]  
  
        ALU(x=Dout, y=AorM, zx=instruction[11], nx=instruction[10],  
            zy=instruction[9], ny=instruction[8], f=instruction[7], no=instruction[6],  
            out=ALUout, zr=zr, ng=ng);  
  
        PC(in=Aout, load=PCload, inc=true, reset=reset, out[0..14]=pc);  
  
        // JUMP condition  
        Or(a=ng, b=zr, out=ngzr);                // ngzr = (ng | zr)  
        Not(in=ngzr, out=g);                      // g = out > 0 = !(ng | zr);  
        And(a=ng, b=instruction[2], out=passLT); // ngLT = (ng & LT)  
        And(a=zr, b=instruction[1], out=passEQ); // zrEQ = (zr & EQ)  
        And(a=g, b=instruction[0], out=passGT);  // gGT = (g & GT)  
        Or(a=passLT, b=passEQ, out=passLE);  
        Or(a=passLE, b=passGT, out=pass);  
  
        And(a=isC, b=pass, out=PCload);           // PCload = isC & J  
  
        // output  
        And16(a=Aout, b=Aout, out[0..14]=addressM);  
        And(a=isC, b=instruction[3], out=writeM); // writeM = isC & d3  
        And16(a=ALUout, b=ALUout, out=outM);  
    }
```

算是硬體部分的重頭戲

然後把記憶體和 HackCPU 合起來 就得到一台電腦了



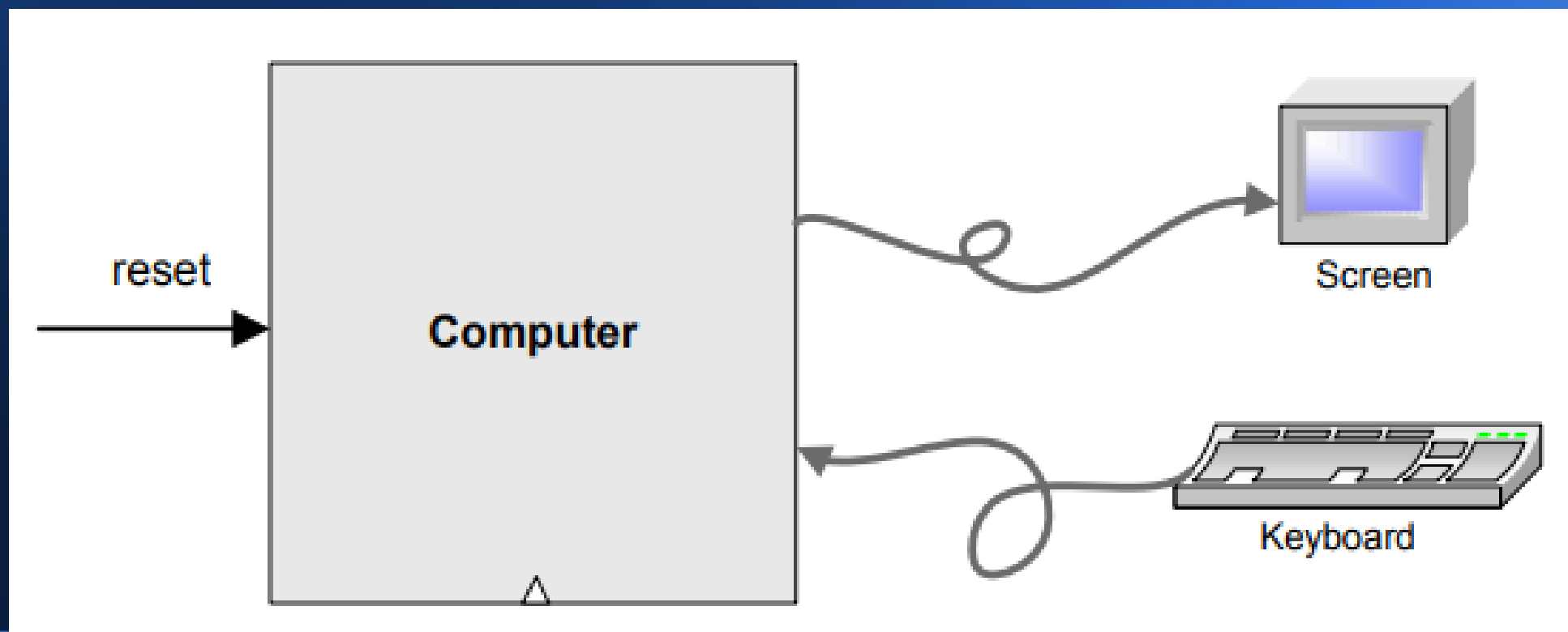
```
CHIP Computer {  
  IN reset;  
  
  PARTS:  
  
  ROM32K(address=pc, out=instruction);  
  CPU(inM=outM, instruction=instruction, reset=reset, outM=inM, writeM=loadM, addressM=addressM, pc=pc);  
  Memory(in=inM, load=loadM, address=addressM, out=outM);  
}
```


但是要設計 HackCPU 之前

- 你必須先瞭解 HackCPU 的組合語言

還要理解記憶體映射輸出入的原理

- 因為 HackCPU 是透過記憶體存取來和鍵盤與螢幕溝通的



這樣

- 寫完 Nand2tetris 的習題後
- 您就擁有設計 CPU 與電腦的經驗了

然後

- 我們今天要講的東西
才真正要開始！

因為我們的標題是

用十分鐘瞭解如何設計電腦

還有讓電腦變快的那些方法

陳鍾誠 2017 年 12 月 7 日

有了以上的背景

- 我們應該可以理解

一台電腦與 CPU 是如何運作的！

但問題是

- 以上那台電腦會很貴 ...
- 因為記憶體是用暫存器做的

如果把記憶體換成慢速的 DRAM

- 那麼整台電腦的速度還會下降幾十倍

而且由於記憶體只有 $24K+1$ 字組

- 所以容量也很小，只能做為
嵌入式的小控制器 ...

於是我們應該想辦法

- 用 DRAM 讓電腦記憶體變大

但是成本不能增加太多

而且還要盡可能的加速.....

HackCPU 另外還有一些速度上的缺陷

- 首先是沒有乘法和除法的硬體電路
 - 於是要用組合語言寫副程式來做乘法和除法
 - 這會讓《乘除法》速度慢上一百倍
- 其次是沒有浮點數的硬體電路
 - 同樣要用組合語言副程式做浮點加減乘除
 - 這樣比起有浮點硬體的 CPU，浮點運算速度恐怕慢上不只百倍了。

乘法器可以用《加法器 + 移位運算》完成

- 以下 VerilogHDL 硬體描述語言用 3 組加法器 + 3 個移位運算完成兩個 4 位元整數的乘法

```
module multiplier(a,b, ab);  
input [3:0] a,b;  
output [7:0] ab;  
  
wire [3:0] t0,t1,t2,t3;  
  
assign t0 = (b[0]==1) ? a : 4'h0;  
assign t1 = (b[1]==1) ? a : 4'h0;  
assign t2 = (b[2]==1) ? a : 4'h0;  
assign t3 = (b[3]==1) ? a : 4'h0;  
  
assign ab=t0+(t1<<1)+(t2<<2)+(t3<<3);  
  
endmodule
```

除法同樣可以用

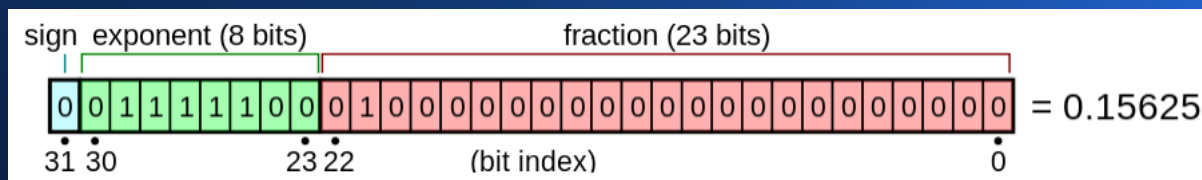
- 減法器 + 移位運算完成
- 就像你做小學《直式除法》時那樣

如果要實作浮點運算

- 必須先理解浮點數格式，目前常用的有 32 位元和 64 位元兩種標準

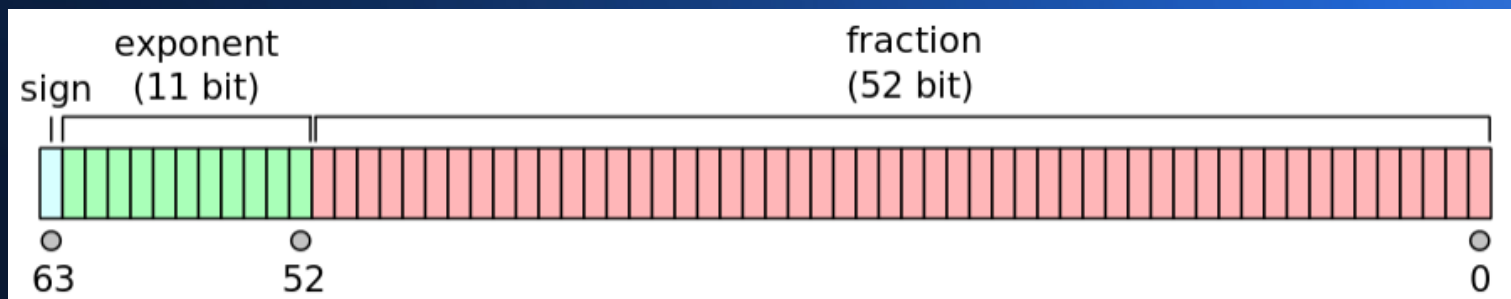
32 位元

單精度浮點數



64 位元

雙精度浮點數



浮點數乘法比加法簡單

- 因為加法要進行《根據指數移位並對齊》的動作，但是乘法不用。

兩個浮點數 x, y

$$x = 2^{E_x} \cdot M_x$$

$$y = 2^{E_y} \cdot M_y$$

加減法公式

$$x \pm y = (M_x 2^{E_x - E_y} \pm M_y) 2^{E_y}, \quad E_x \leq E_y$$

乘法公式

$$x \times y = 2^{(E_x + E_y)} \cdot (S_x \times S_y)$$

除法公式

$$x \div y = 2^{(E_x - E_y)} \cdot (S_x \div S_y)$$

以下是 27 位元 浮點數乘法之 Verilog 程式

```
module FpMul (input [26:0] iA, [26:0] iB, output [26:0] oProd);
    // ... 變數宣告
    assign oProd_s = A_s ^ B_s; // XOR sign bits to determine product sign.

    // Multiply the fractions of A and B
    assign pre_prod_frac = A_f * B_f;

    assign pre_prod_exp = A_e + B_e;    // Add exponents of A and B

    // If top bit of product frac is 0, shift left one
    assign oProd_e = pre_prod_frac[35] ? pre_prod_exp : (pre_prod_exp - 8'b1);
    assign oProd_f = pre_prod_frac[35] ? pre_prod_frac[35:18] : pre_prod_frac[34:17];

    assign underflow = A_e[7] & B_e[7] & ~oProd_e[7];    // Detect underflow

    // Detect zero conditions (either product frac doesn't start with 1, or underflow)
    assign oProd = ~oProd_f[17] ? 27'b0 :
        underflow      ? 27'b0 :
        {oProd_s, oProd_e, oProd_f};
endmodule
```


浮點加法比乘法複雜

```
module FpAdd (input iCLK, [26:0] iA, [26:0] iB, output [26:0] oSum);
    // 變數宣告，省略 ....
    assign exp_diff_A = {B_e[7], B_e} - {A_e[7], A_e};
    assign exp_diff_B = {A_e[7], A_e} - {B_e[7], B_e};
    assign larger_exp = exp_diff_B[8] ? B_e : A_e;
    assign A_f_shifted = exp_diff_A[8] ? {1'b0, A_f, 18'b0} :
        (exp_diff_A > 9'd35) ? 37'b0 :
        ({1'b0, A_f, 18'b0} >> exp_diff_A);
    assign B_f_shifted = exp_diff_B[8] ? {1'b0, B_f, 18'b0} :
        (exp_diff_B > 9'd35) ? 37'b0 :
        ({1'b0, B_f, 18'b0} >> exp_diff_B);
    // Determine which of A, B is larger
    assign A_larger = exp_diff_A[8] | (~exp_diff_B[8] & (A_f > B_f));
    // Calculate sum or difference of shifted fractions.
    assign pre_sum = ((A_s^B_s) & A_larger) ? A_f_shifted - B_f_shifted :
        ((A_s^B_s) & ~A_larger) ? B_f_shifted - A_f_shifted :
        A_f_shifted + B_f_shifted;
    always @(posedge iCLK) begin
        buf_pre_sum    <= pre_sum;
        buf_larger_exp <= larger_exp;
        buf_A_f_zero   <= (A_f == 18'b0);
        buf_B_f_zero   <= (B_f == 18'b0);
        buf_A           <= iA;
        buf_B           <= iB;
        buf_oSum_s      <= A_larger ? A_s : B_s;
    end
end
```

(浮點加法第一頁程式碼)

因為需要執行對齊動作

```
// Determine output fraction and exponent change with position of first 1.
assign shft_amt = pre_frac[36] ? 8'd0 : pre_frac[35] ? 8'd1 :
                 pre_frac[34] ? 8'd2 : pre_frac[33] ? 8'd3 :
                 // 省略 ...
                 pre_frac[4] ? 8'd32 : pre_frac[3] ? 8'd33 :
                 pre_frac[2] ? 8'd34 : pre_frac[1] ? 8'd35 :
                 pre_frac[0] ? 8'd36 : 8'd37;

assign pre_frac_shft = {pre_frac, 17'b0} << shft_amt;
assign oSum_f = pre_frac_shft[53:36];

assign oSum_e = buf_larger_exp - shft_amt + 8'b1;

// Detect underflow
wire underflow;
assign underflow = ~oSum_e[7] && buf_larger_exp[7] && (shft_amt != 8'b0);

assign oSum = buf_A_f_zero ? buf_B :
              buf_B_f_zero ? buf_A :
              (pre_frac == 38'b0) ? 27'b0 :
              underflow ? 27'b0 :
              {buf_oSum_s, oSum_e, oSum_f};

endmodule
```

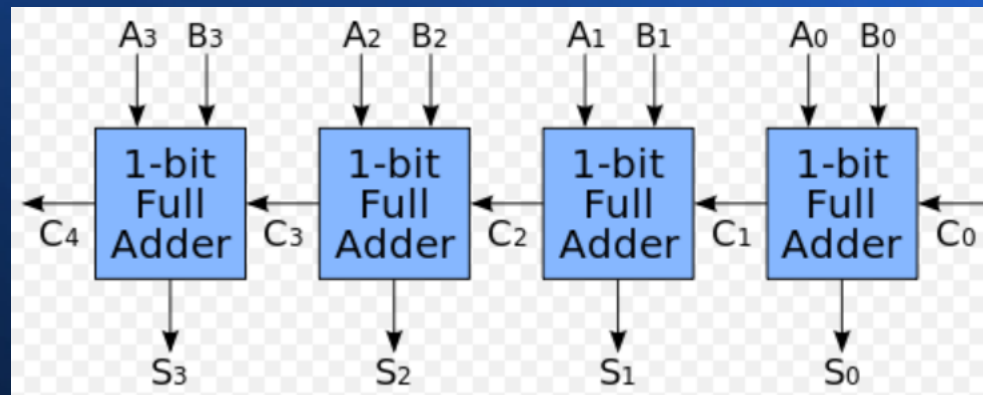
(浮點加法第二頁程式碼)

有了整數的乘除法電路

- 還有浮點運算電路之後
 - CPU 的數學運算差不多就完整了
 - 最好還能加入移位運算
- 這樣差不多就是完整的處理器了

但是、還有一個速度問題

- HackCPU 的 ALU 使用的是《鏈波進位加法器》，速度太慢



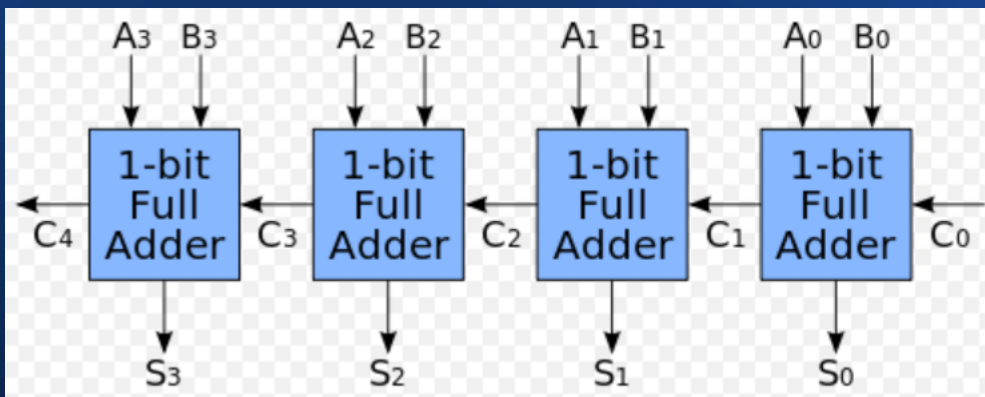
- 16 位元處理器其電子《經過的閘數》會是上圖的四倍
因為必須串接 16 個全加器
- 這種鏈波方式是前一個運算完後電子才會到達下一個，因此要經過 16 層之後才會算完。

因此 HackCPU 當然就會太慢

- 不過我們可以改進其速度
- 只要用《前瞻進位加法器》
取代《鏈波進位加法器》
- 速度就會快上數倍

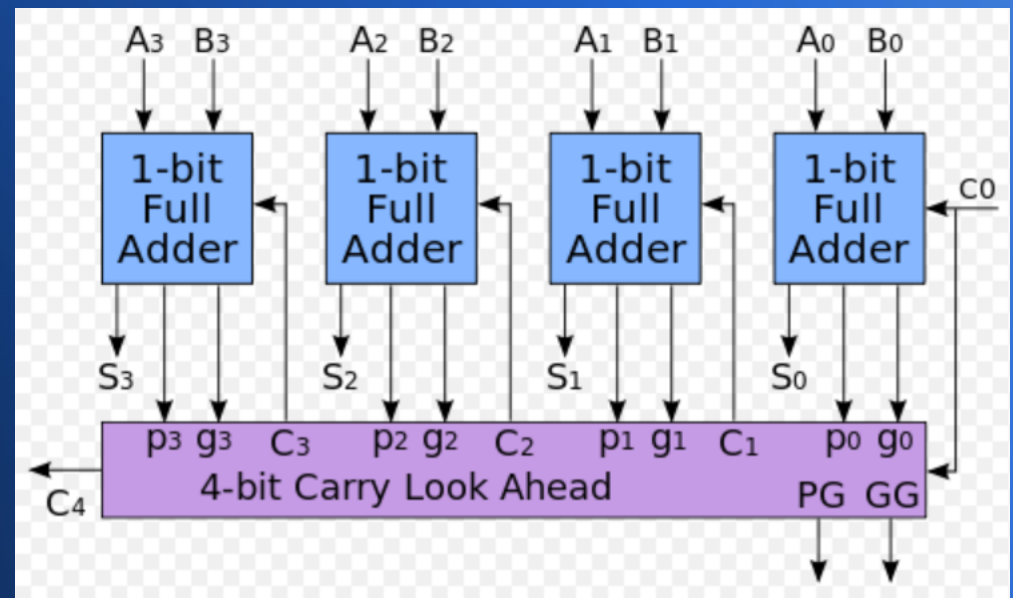
以下是這兩種加法器的結構圖

鏈波進位加法器



粉紅色區塊就像一個快速通道
可以快速計算出 c_1, c_2, c_3 之後
丟給對應的全加器去算出結果。

前瞻進位加法器



快速的前瞻進位加法器

- 是 IBM 的 Gerald Rosenberger 在 1957 年申請的專利，該專利在 1977 年就過時了，因此可以自由的使用。
- 其設計依賴下列遞迴公式

$$G(A, B) = A \cdot B$$

$$P(A, B) = A + B$$

$$C_{i+1} = G_i + (P_i \cdot C_i)$$

$$C_1 = G_0 + P_0 \cdot C_0 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

$$C_3 = G_2 + P_2 \cdot C_2 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

$$C_4 = G_3 + P_3 \cdot C_3 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$

這樣就能讓 ALU 更快

- 進而讓 CPU 變得更快

如果完成上面的改良

- 那麼這顆 CPU 就會是一顆
《速度較正常》的 CPU 了

但是絕對稱不上快版

- 而且比起現代的 CPU 而言，仍然算是極慢的.....

因為現代的 CPU

- 通常有下列加速技巧
 - 多層次快取 ...
 - 管線 pipeline 機制 ...
 - 多核心 + Hyper-Threading ...
 - 螢幕繪圖交給顯卡上的 GPU

一般來說

- 常見的加速方法有兩種...

哪兩種？

- 一種是用快的材料取代慢的
- 另一種是平行，也就是用很多個一起工作讓速度變快.....

首先讓我們來看如何用《快取》加速

- 多層次快取 ... 
- 管線 pipeline 機制 ...
- 多核心 + Hyper-Threading ...
- 螢幕繪圖交給顯卡上的 GPU

記憶體階層架構

- 就是盡量在不大幅增加成本的情況下
用《快取》加快速度的策略 ...

我們先前曾經看到這頁

記憶體階層是在電腦架構下儲存系統階層的排列順序。每一層於下一層相比都擁有較高的速度和較低延遲性，以及較小的容量（也有少量例外，如AMD早期的Duron CPU）。大部分現今的中央處理器的速度都非常的快。大部分程式工作量需要記憶體存取。由於快取的效率和記憶體傳輸位於階層中的不同等級，所以實際上會限制處理的速度，導致中央處理器花費大量的時間等待記憶體I/O完成工作。

大部分電腦中的記憶體階層如下四層：

1) 暫存器—可能是最快的存取。在32位元處理器，每個暫存器就是32位元。x86處理器共有16個暫存器。

2) 快取（L1-L3: SRAM）

第一級快取（L1）—通常存取只需要幾個週期，通常是幾十個KB。

第二級快取（L2）—比L1約有2到10倍較高延遲性，通常是幾百個KB或更多。

第三級快取（L3）（不一定有）—比L2更高的延遲性，通常有數MB之大。

第四級快取（L4）（不普遍）—CPU外部的DRAM，但速度較主記憶體高。

3) 主記憶體（DRAM）—存取需要幾百個週期，可以大到數十GB。

4) 磁碟儲存—需要成千上百個週期，容量非常大。

加入適當容量的快取

- 可以讓電腦在不會貴太多的情況下還能變快很多 ...

現代的電腦

- 快取特別重要
- 因為大容量的 DRAM 記憶體速度，
已經比 CPU 的暫存器速度
《慢上百倍》...

1970 年代

- DRAM 記憶體速度只比 CPU 內的暫存器慢兩到三倍...
- 但是 CPU 內暫存器的速度每年提升得比記憶體快很多
(有些年份是 50% v. s. 10%)

結果相對於 CPU 而言

- 記憶體變成一個《很慢》的儲存裝置

如果不採用快取

- 整個電腦的運作速度就得向記憶體看齊，那就會慢得不得了

但是加了快取之後

- 如果指令執行時，總是存取那些不在快取中的資料，那快取就無效了

所以除了快取設計得好之外

- 程式存取記憶體時，也要能讓
《快取命中率》夠大才行

快取通常採用區域性策略

- 一次載入一整塊
- 所以如果程式的區域性不好，快取失誤就會很多。

像是下列兩個程式

- 雖然只差一點點（ i, j 順序調換），但是區域性可是差異很大的
- 上面的程式大部分都快取命中
- 下面的程式幾乎每次都會快取失誤

```
// 區域性好的程式
for (i=0; i<100000000; i++)
    for (j=0; j<100000000; j++)
        m[i][j] = a[i][j]
```

```
// 區域性不好的程式
for (j=0; j<100000000; j++)
    for (i=0; i<100000000; i++)
        m[i][j] = a[i][j]
```

但是儘管記憶體很慢

- 不過和硬碟比起來，記憶體還算是很快的。
- 尤其是硬碟讀寫頭要旋轉到位才可以開始讀，那個機械動作（比起電子動作而言）更是慢得不得了...

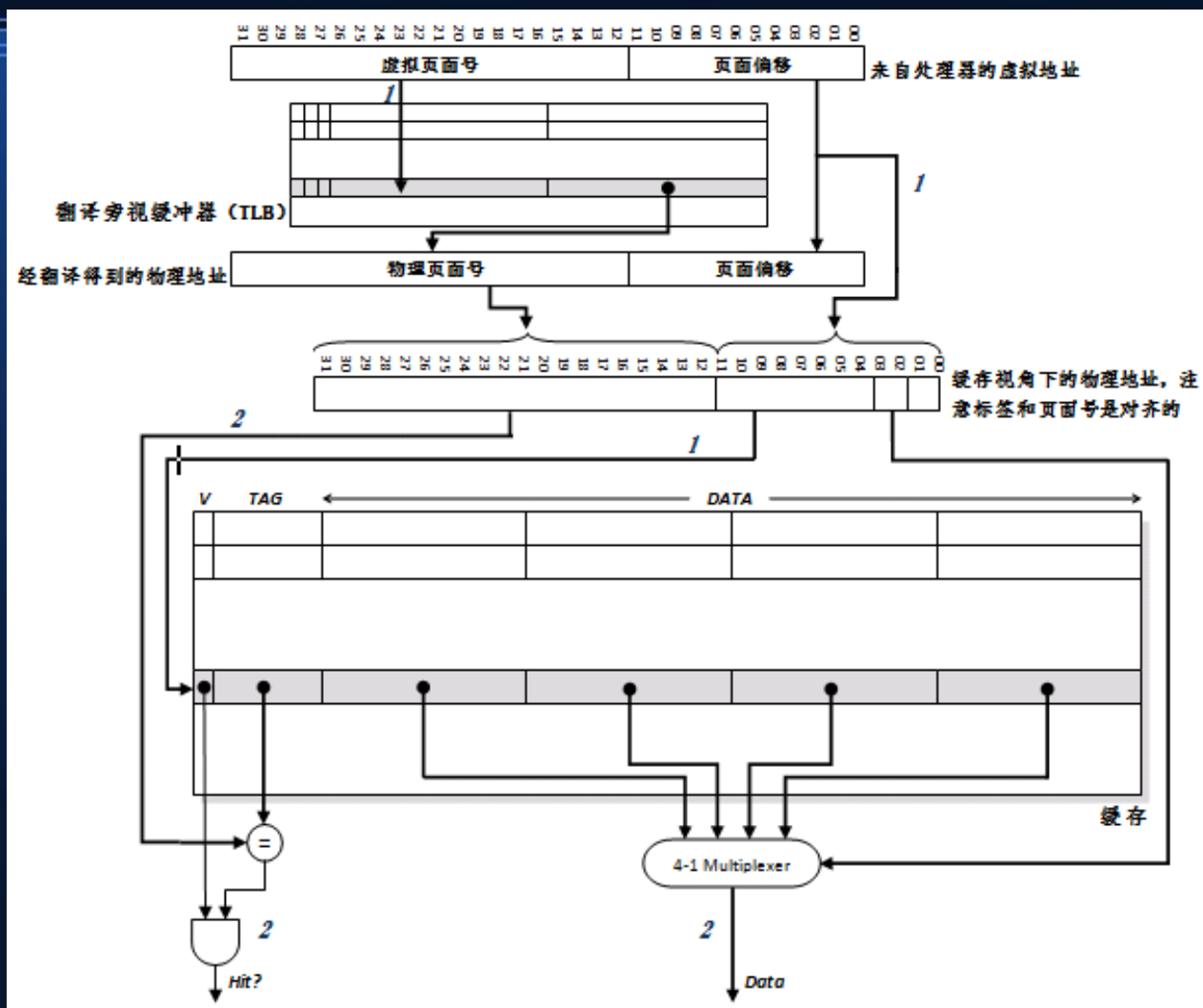
所以

- 沒有最慢，只有更慢 ...
- 沒有最快，只有更快 ...

只要盡量減少慢速裝置的存取

- 改用小量快速裝置當快取，就能讓電腦的效能有顯著的提升

當然快取也需要電路才能完成



以下是 AMD Bulldozer 處理器的架構

- 每個核心內部都有三層快取



CPU

- 可以透過快取減少存取慢速元件的機會
- 進而讓效能不會被慢速元件卡住...

接著讓我們看看第二類加速方法

- 也就是《平行機制》...

平行

- 是利用多個元件同時執行，達到加速的目的

今日的電腦

- 通常採用下列三種平行機制
 - 管線 pipeline 機制 ...
 - 多核心 + Hyper-Threading ...
 - 螢幕繪圖交給顯卡上的 GPU

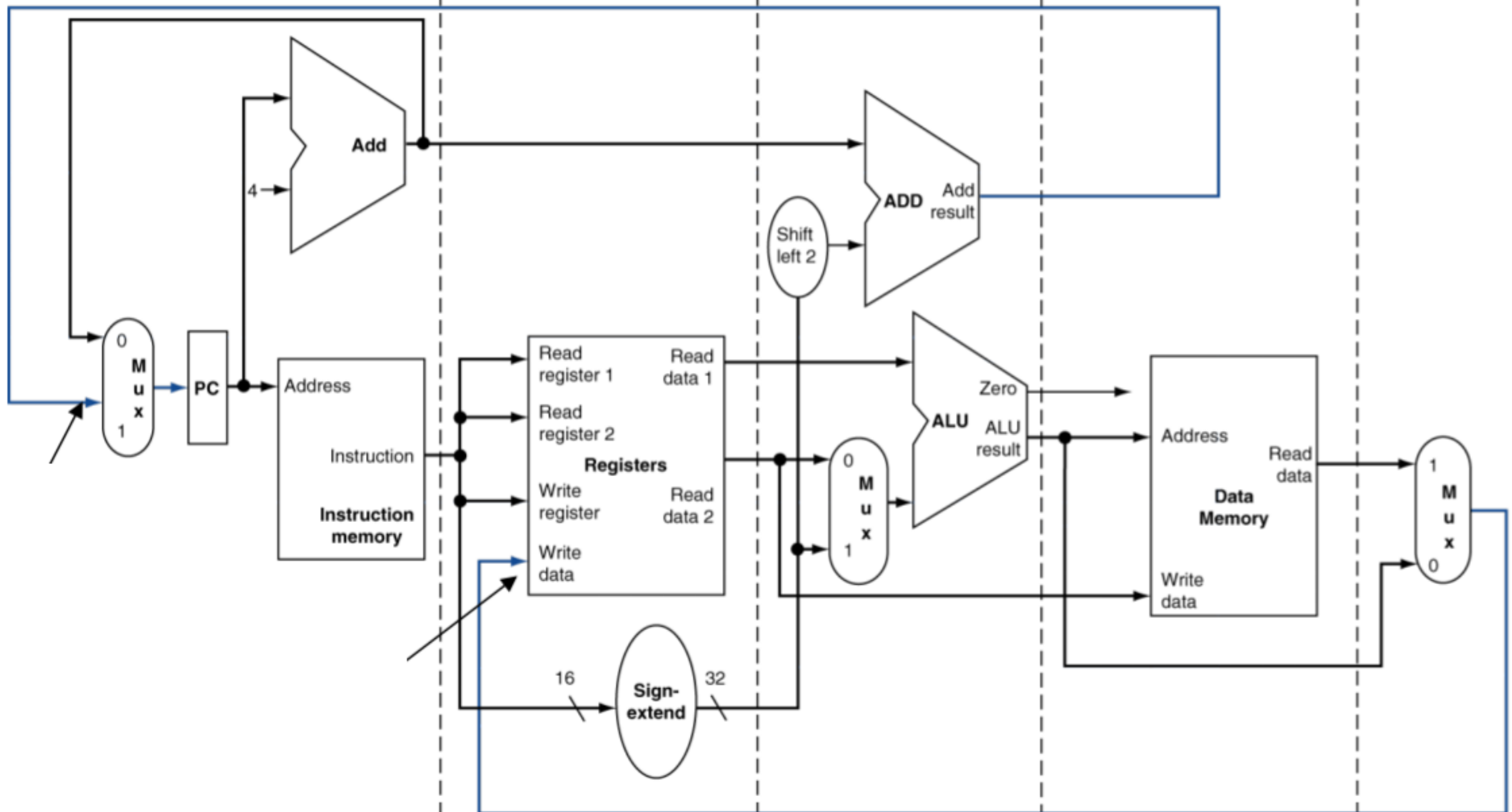
首先讓我們介紹

- 管線 pipeline 平行機制...

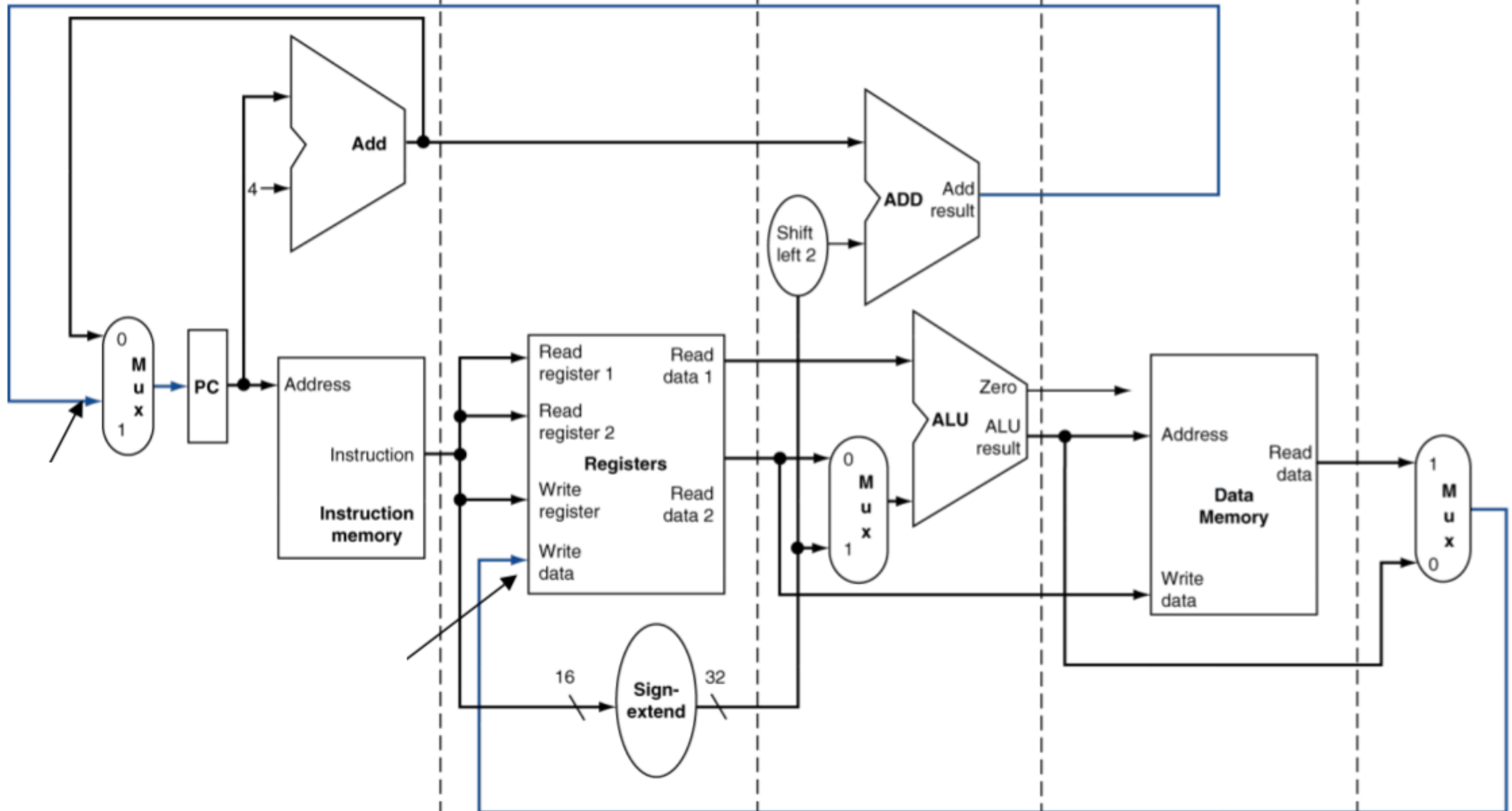
要理解 pipeline

- 首先要理解傳統處理器的結構

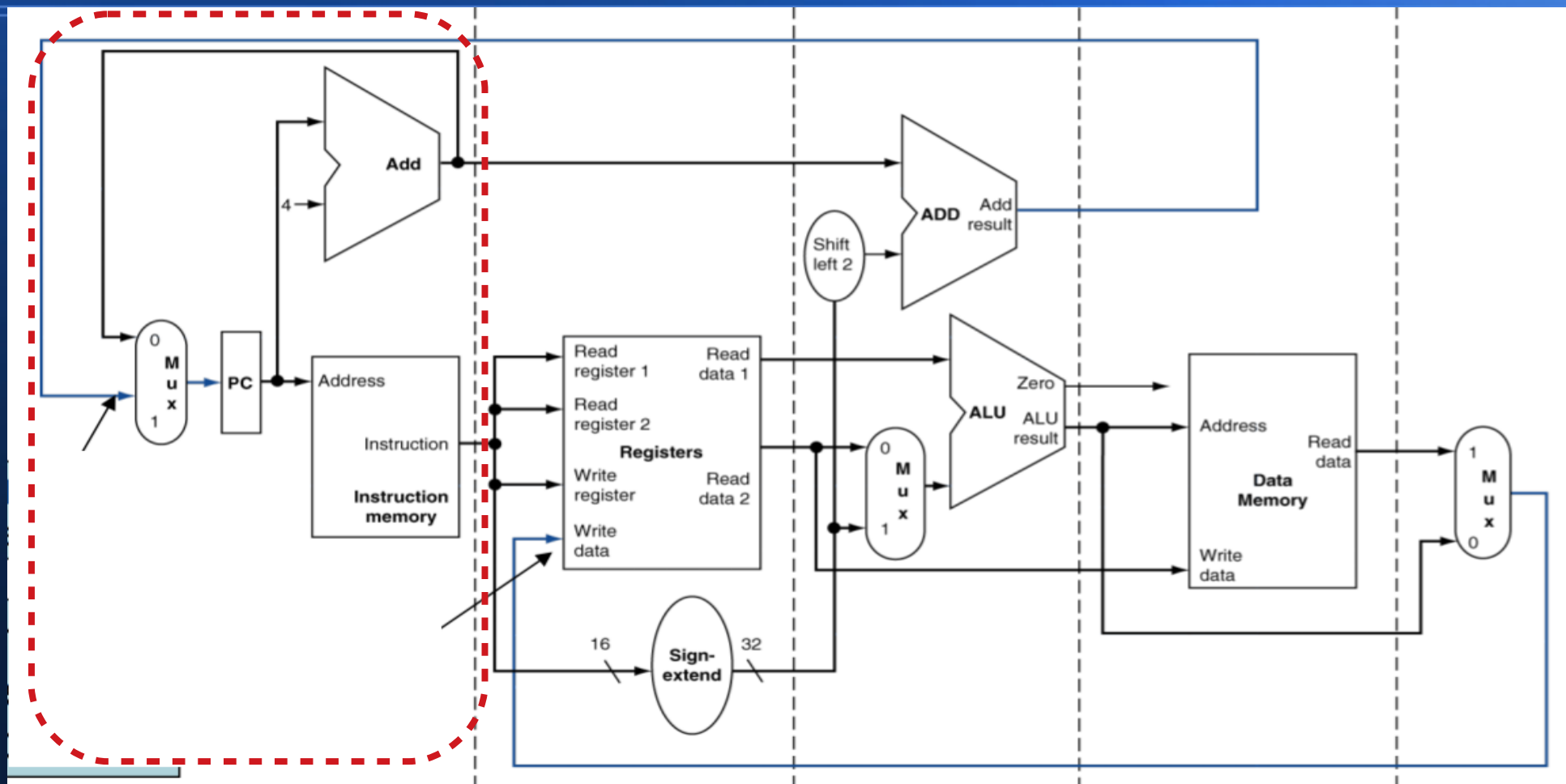
以下是 MIPS CPU 的簡化圖



虛線將 CPU 分成五個區塊



在單一時間點，只會有一個區塊在做事



指令擷取 IE 時期

指令解碼 ID 時期

執行 EXE 時期

存取 MEM 時期

寫回 WB 時期

這就像生產線上有五個人

- 分別做《加料、成型、切割、包裝、綁線》等動作...
- 但是每次只有一個產品在線上
- 所以總是有四個人處於閒閒沒事幹的狀態...

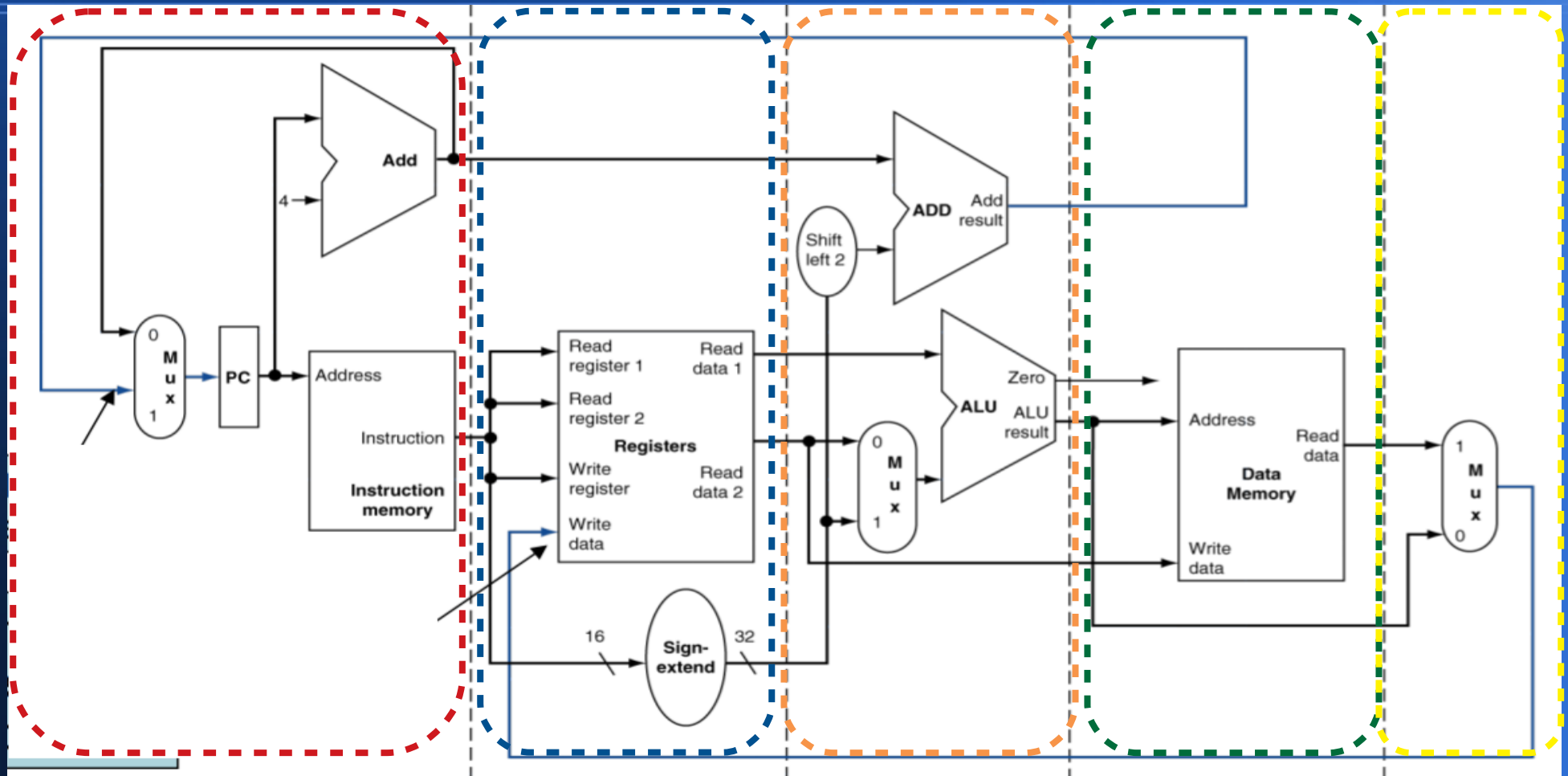
於是我們會想

- 幹嘛不把產品連續丟上生產線，
這樣五個人不就都有事做了 …

像是這樣



也就是讓五個區塊 同時執行不同的動作



指令擷取 IE 時期

指令解碼 ID 時期

執行 EXE 時期

存取 MEM 時期

寫回 WB 時期

只要生產線滿載，就可以五路齊發

Instr No.	Pipeline Stage						
	IF	ID	EX	MEM	WB		
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

RISC機器的五層管線示意圖（IF：讀取指令，ID：指令解碼，EX：執行，MEM：記憶體存取，WB：寫回暫存器）



問題是

- 電子的流動是沒有分區的，除非我們用某種《圍牆》把電子區隔開來...

甚麼樣的元件

- 可以將電流分隔成一區一區的呢？

答案是

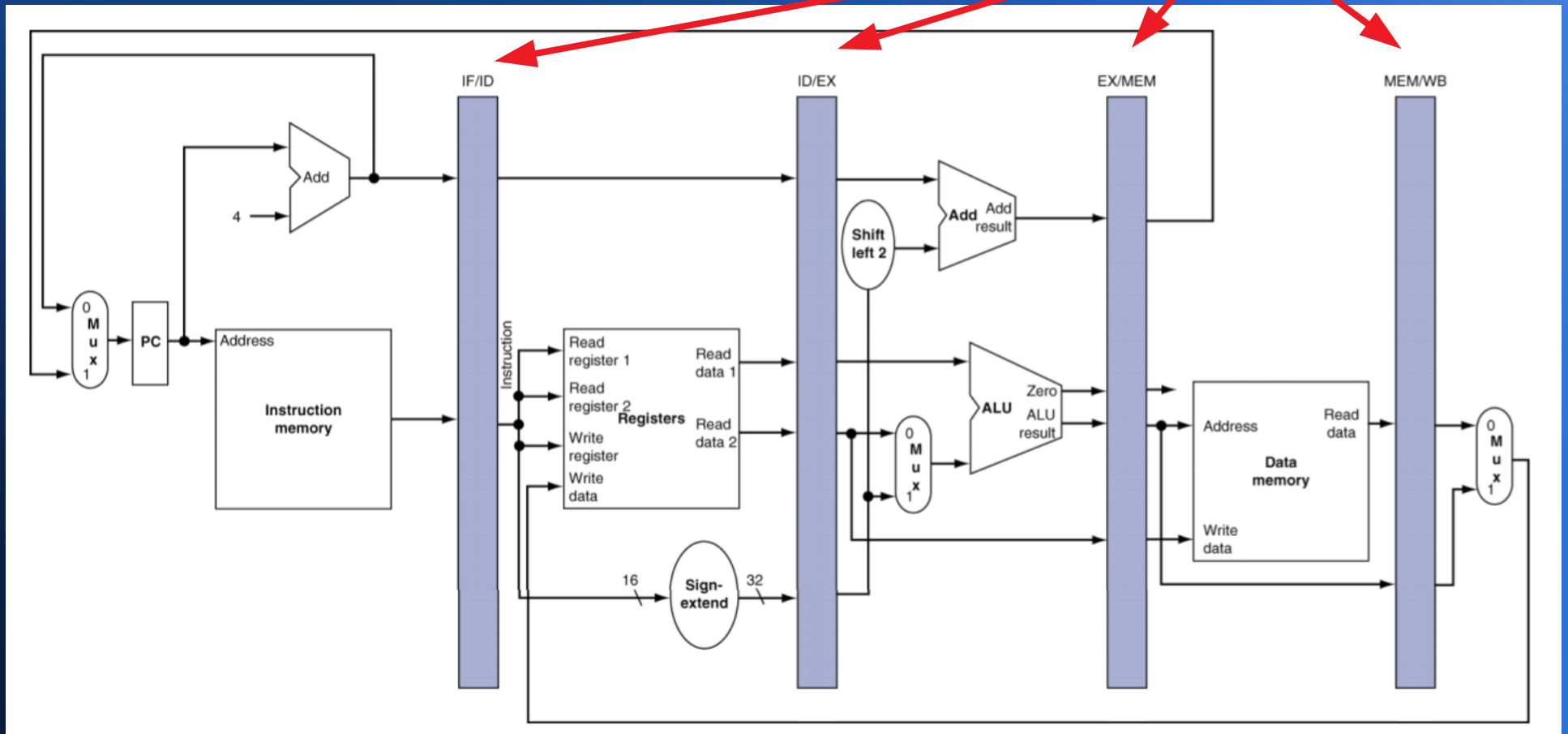
暫存器

因為暫存器

- 是由《邊緣觸發型正反器》所組成的
 - 例如 : DFF (D Flip-Flop)
- 《邊緣觸發型元件》可以扮演圍牆的角色，將電路區隔開來...

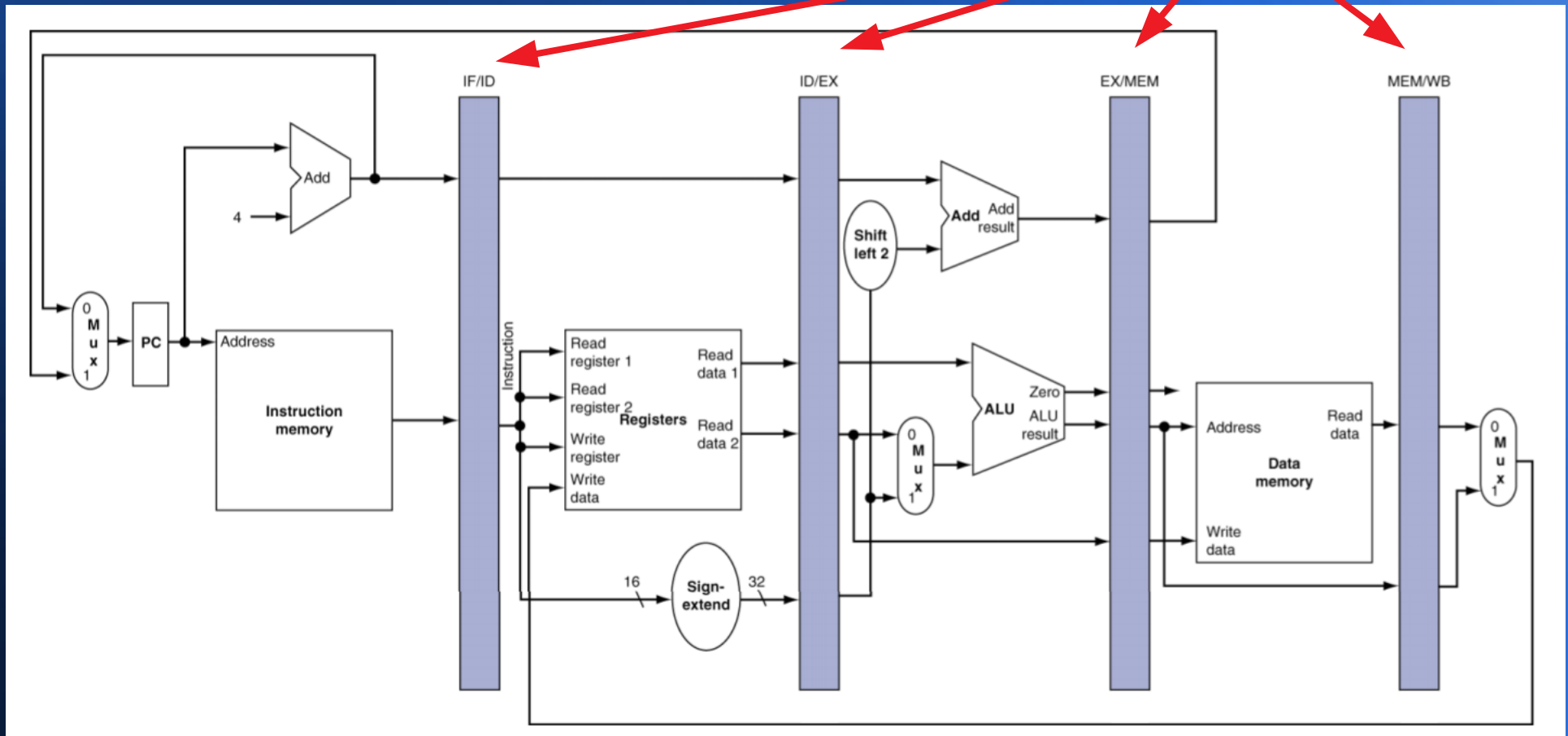
像是這樣

暫存器圍牆



其中的《暫存器》像圍牆一樣把各區域隔開

暫存器圍牆

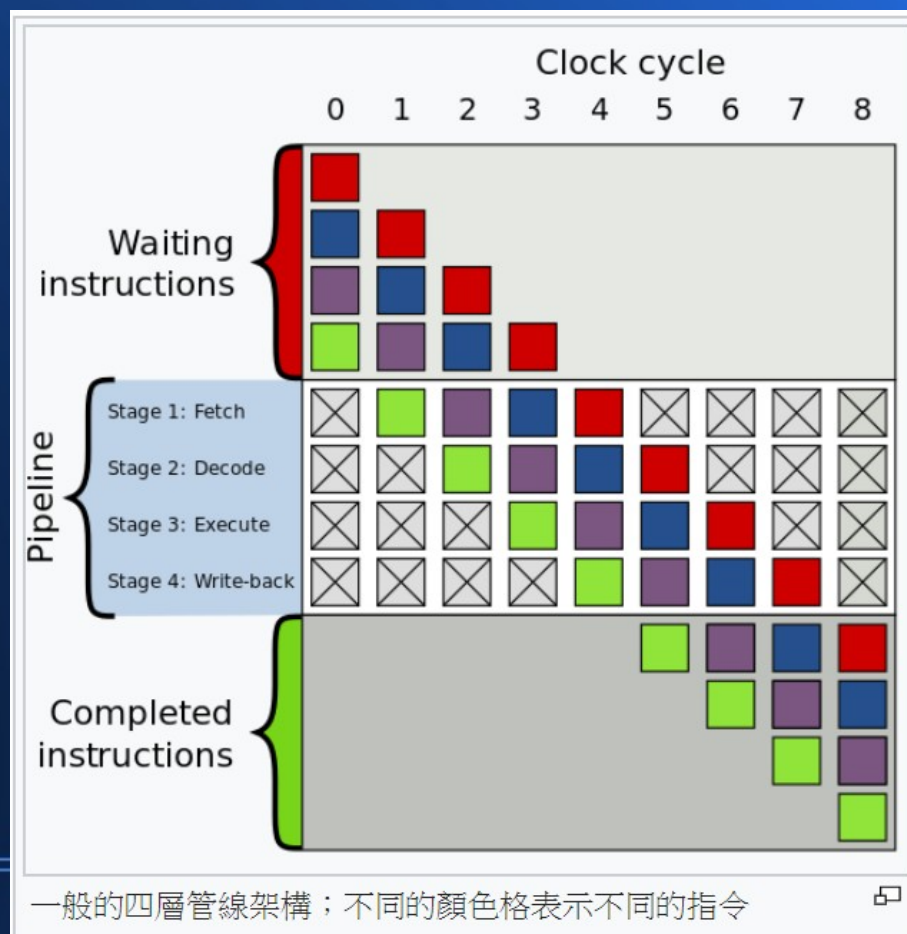


於是五個區塊都可以同時運作

- 不會互相干擾 ...
- 結果是《擷取 IF，解碼 ID、執行 EXE、存取 MEM、寫回 WB》都可以同時執行
- 只是每個區塊執行的是不同指令而已
- 就好像是生產線上的《某個時間點》，每個人處理的《產品》並不是同一個 ...
- 但是每一個產品都會被線上每個員工處理過...

這樣的話

- 如果分成五個階段，那麼速度最多可以提升五倍



現代的處理器

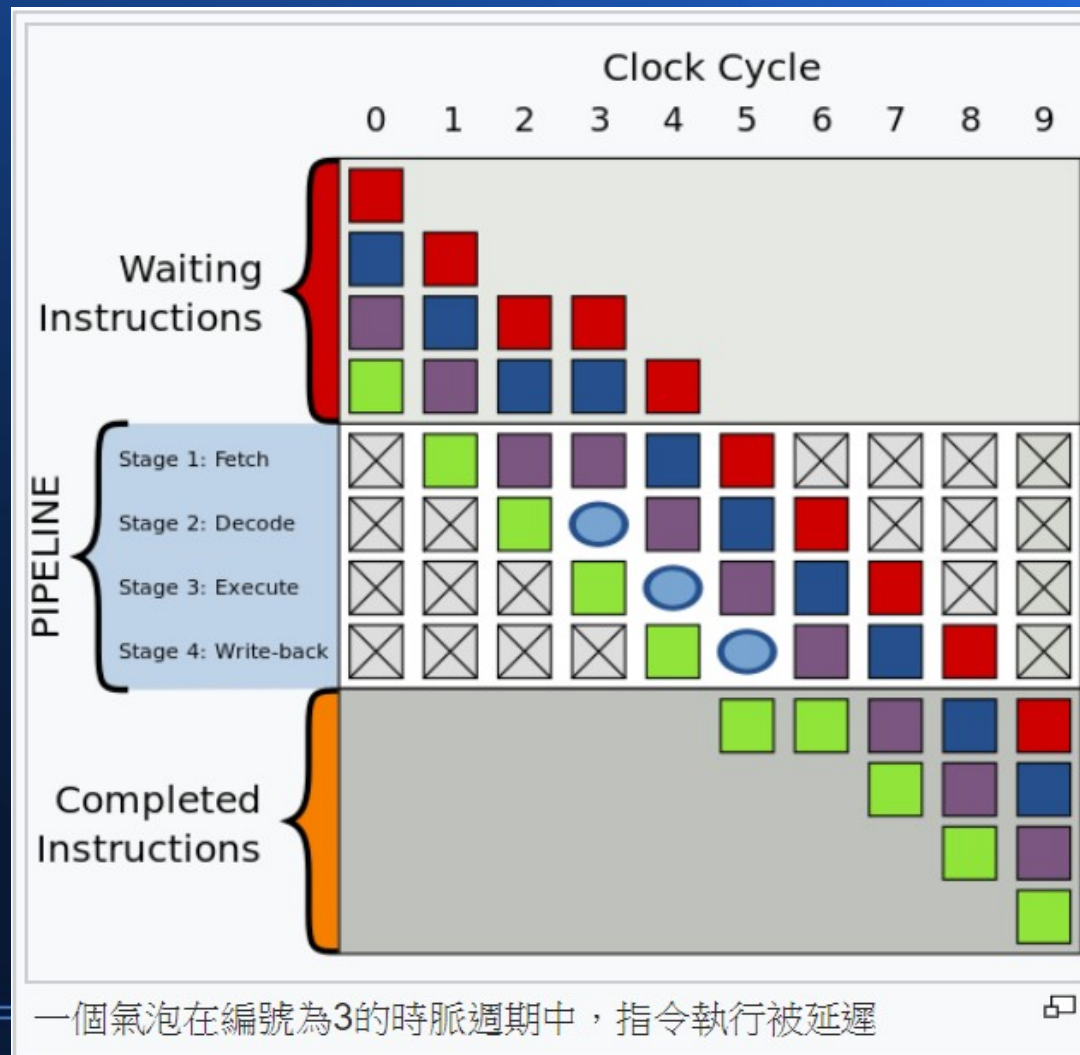
- 一個指令常常會被分成十幾個執行階段，
因此速度提升最高可以達十幾倍.....

但是

- 現實世界沒有那麼簡單
- 管線的平行動作有可能會被跳躍指令打斷，像 JLT 這類的條件跳躍指令會造成已經執行的動作被廢棄或撤回
- 還有記憶體存取時，若資料不在第一級快取中，那麼就無法順暢的執行管線...

這些障礙都會造成《管線泡泡》

讓 CPU 難以全速前進....



以上

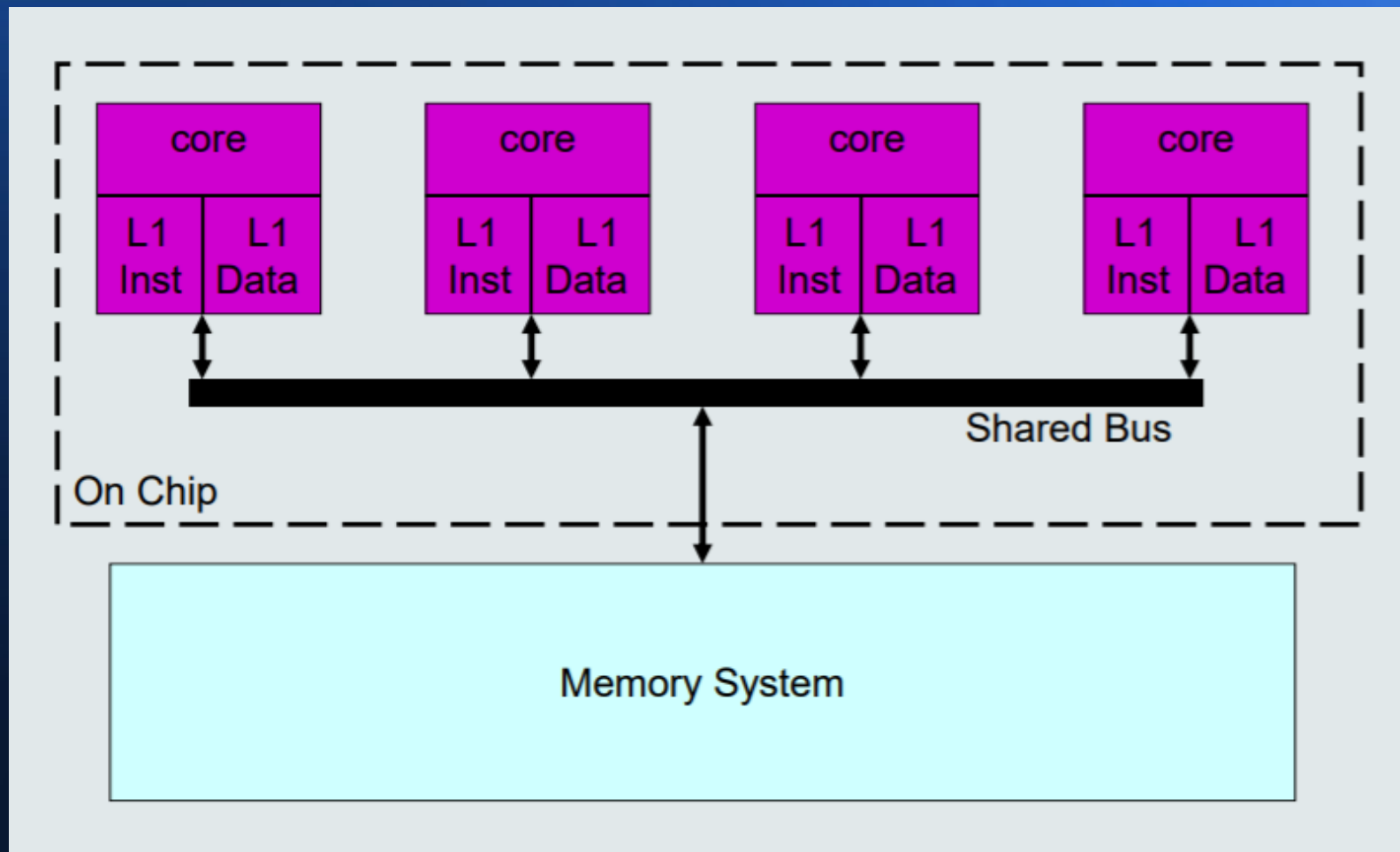
- 就是管線 pipeline 的加速原理

接著讓我們看看《多核心》

- 多層次快取 ...
- 管線 pipeline 機制 ...
- 多核心 + Hyper-Threading ...
- 螢幕繪圖交給顯卡上的 GPU

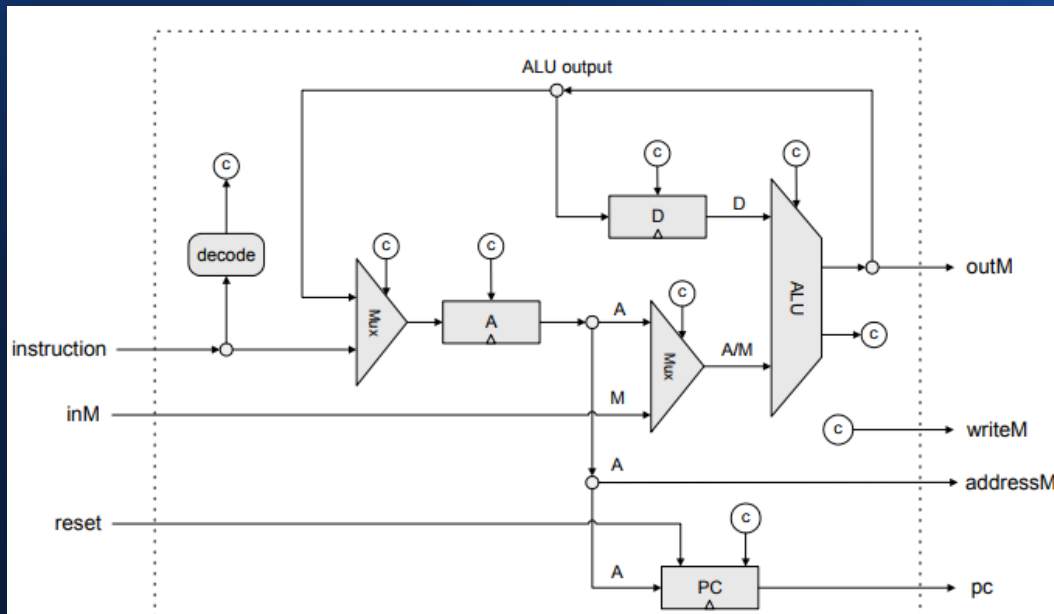


以下是個四核心 CPU 的示意圖



其中每個 core

- 都有自己的 ALU、暫存器、控制電路等等...
- 就像我們前面 nand2tetris 學到的那樣 ...

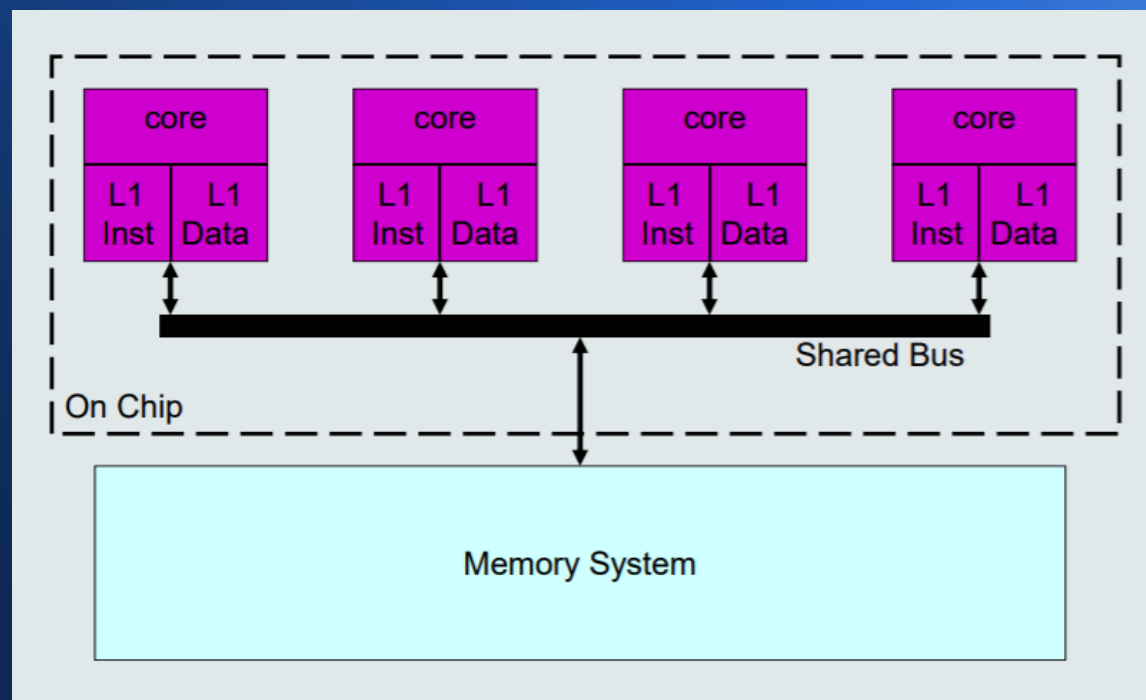


core



每個 core 都可以執行指令

- 所以 4 個 core 最快就可以達到四倍速

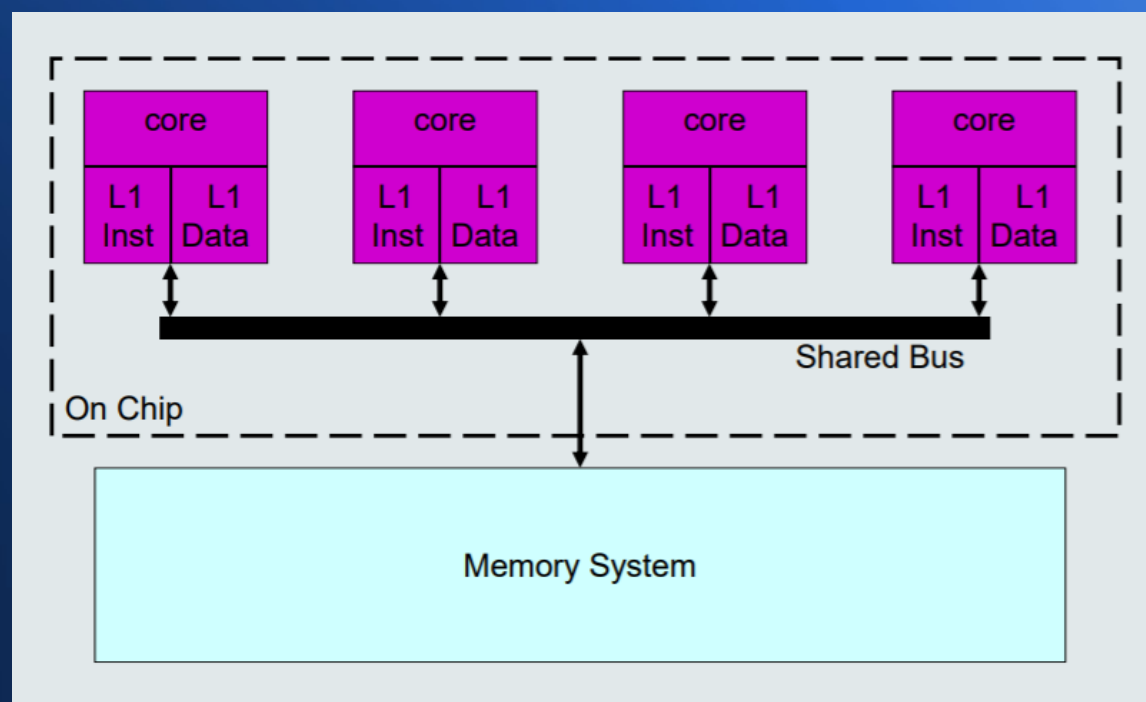


但是大部分的程式

- 都是不能完全平行化的！
- 不過我們只要將《瓶頸》的部分盡可能平行化，就有可能大幅改進效能....

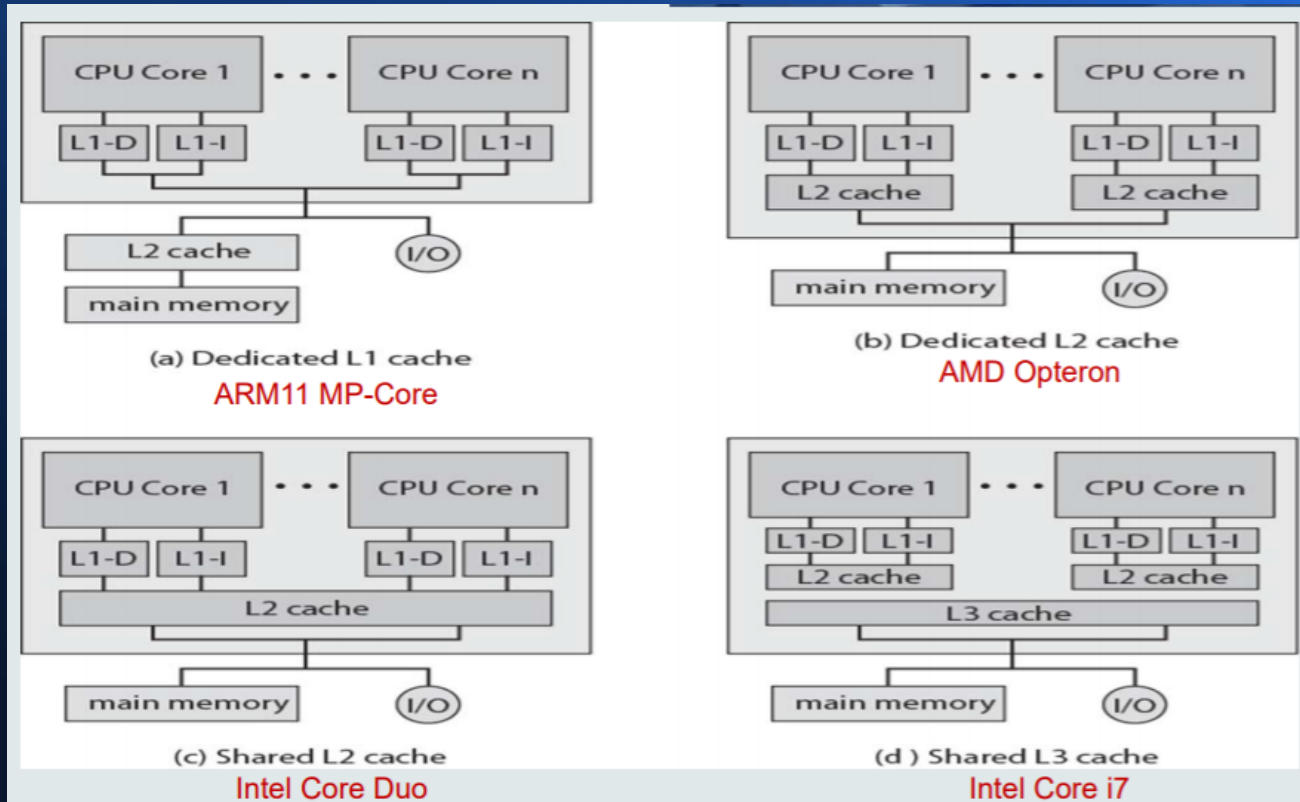
像是由於記憶體共用

- 所以同時只能有一個 core 存取記憶體
- 因此速度通常無法達到四倍



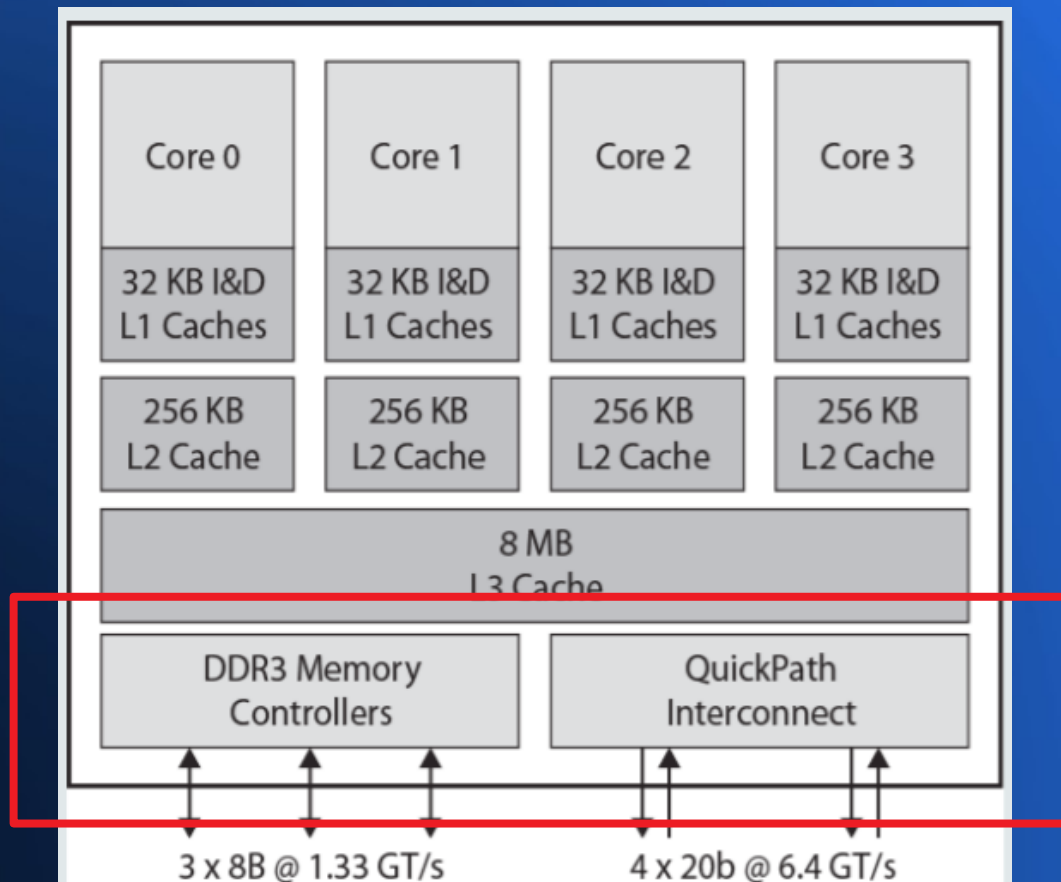
有些多核心的 core

- 會把 L2, L3 快取也包進來



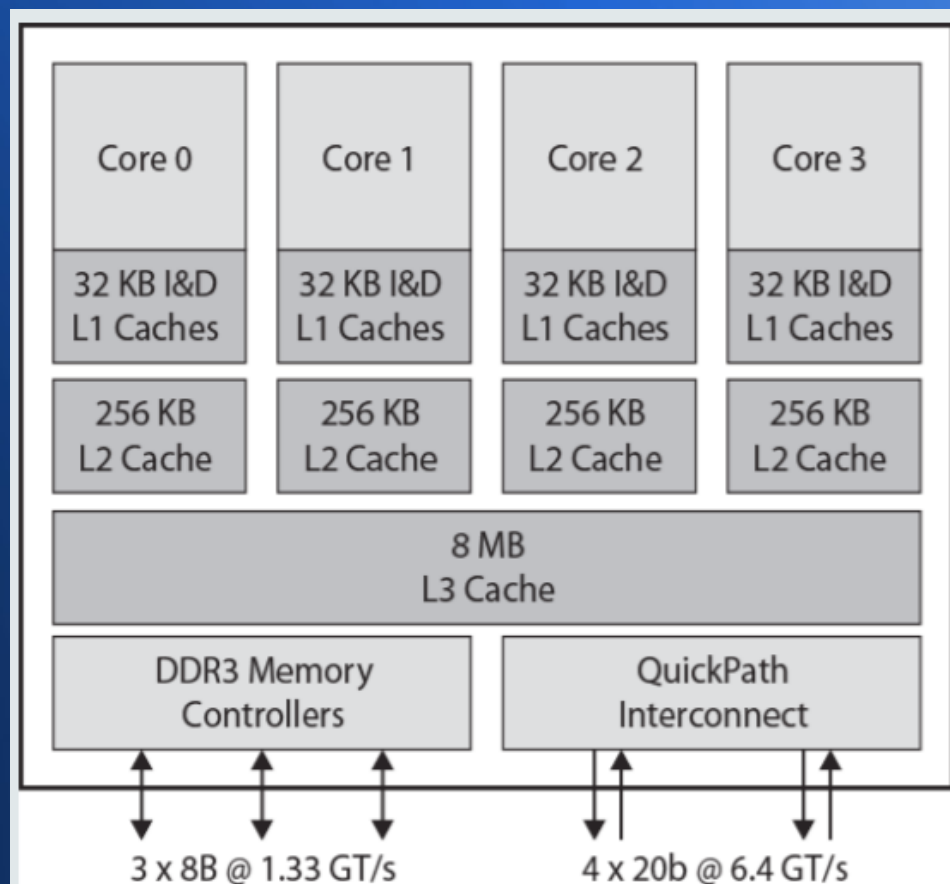
下圖是 Intel Core i7 的四核心架構

資料存取的控制區塊



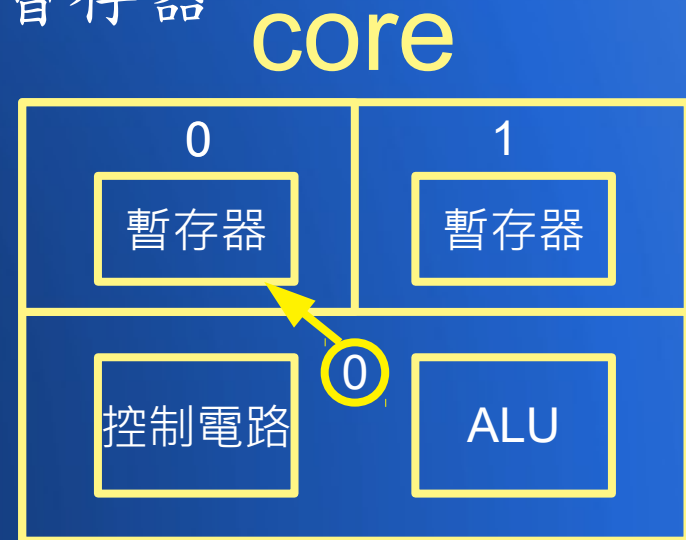
理想上、一個 core 執行一個 thread 最能全速發揮

- 但是萬一 thread 進入 I/O 狀態，或者快取失誤而必須存取記憶體
- 這時候該 core 就會停下來了



聰明的 CPU 設計者

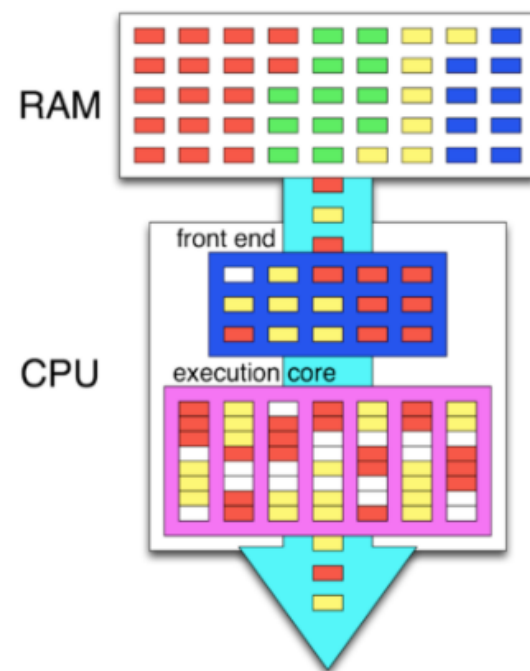
- 於是決定讓一個 core 上有兩套暫存器與相關電路。
- 這樣就能讓一個 core 上面可以跑兩個 thread，而且不需要在切換時重新由記憶體載入暫存器
- 因為重新載入暫存器至少得花上數十個記憶體存取週期
- 這種設計讓 thread 間的切換可以在一兩個 clock cycle 內就切換過來



這就是所謂的 Hyper-Threading 技術

超執行緒（HT, Hyper-Threading）^[1]是英特爾研發的一種技術，於2002年發布。超執行緒技術原先只應用於Xeon 處理器中，當時稱為「Super-Threading」。之後陸續應用在Pentium 4 HT 中。早期代號為Jackson。

通過此技術，英特爾實現在一個實體CPU中，提供兩個邏輯執行緒。之後的Pentium D縱使不支援超執行緒技術，但就整合了兩個實體核心，所以仍會見到兩個執行緒。超執行緒的未來發展，是提升處理器的邏輯執行緒。英特爾於2016年發布的Core i7-6950X便是將10核心的處理器，加上超執行緒技術，使之成為20個邏輯執行緒的產品。



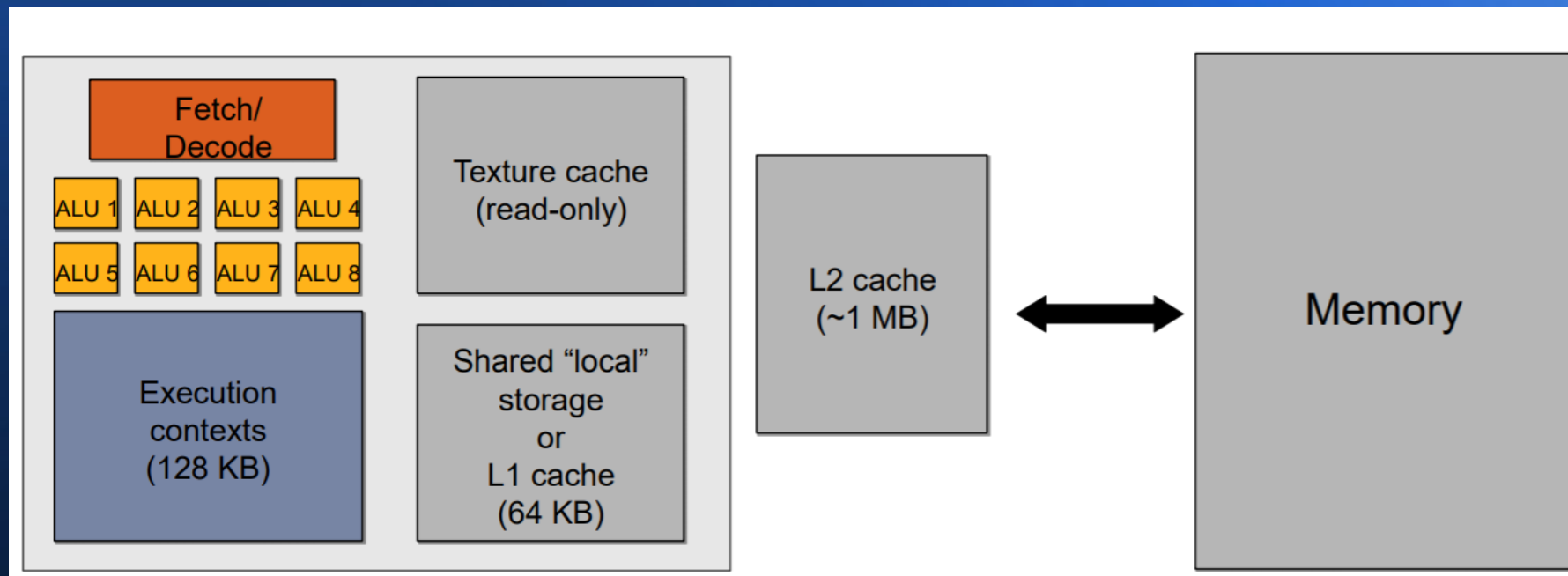
接下來讓我們看看 GPU

- 多層次快取 ...
- 管線 pipeline 機制 ...
- 多核心 + Hyper-Threading ...
- 螢幕繪圖交給顯卡上的 GPU 

GPU 繪圖處理器

- 通常具有上百個浮點乘法與加法的運算單元，於是對這類運算會比 CPU 快很多
- 螢幕繪製的動作，通常就需要很多這類運算，所以在繪圖上 GPU 有可能比 CPU 快上數十倍甚至上百倍。

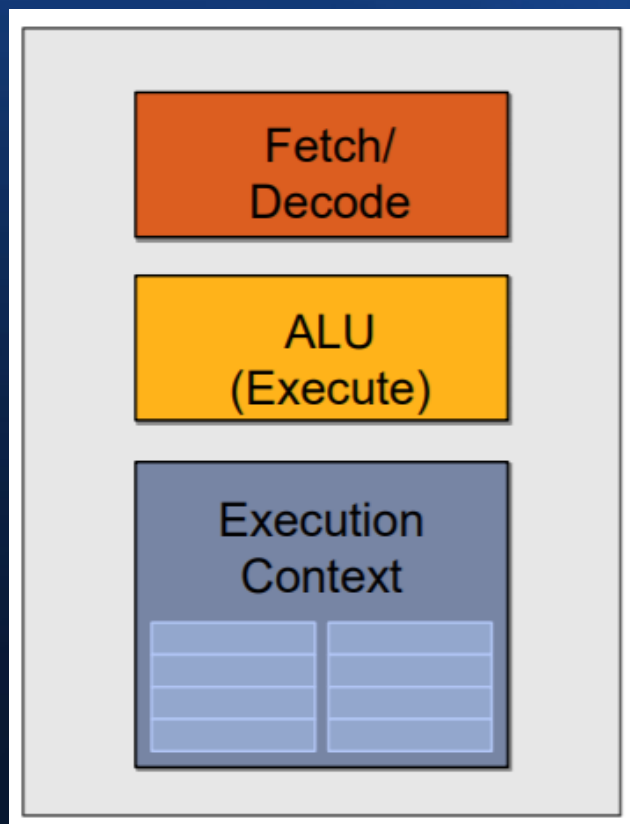
以下是一個包含 8 個 ALU 的 GPU 架構



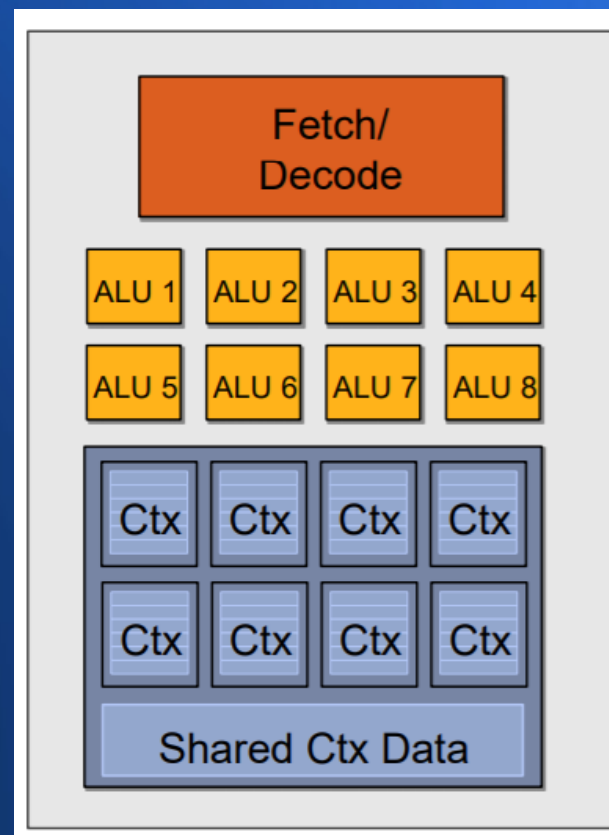
對照一下 CPU 與 GPU 兩者

會發現 GPU 裡有很多 ALU 單元

CPU 的 core



GPU 的 core



兩者的程式碼也是不同的

```
<diffuseShader>:  
sample r0, v4, t0, s0  
mul   r3, v0, cb0[0]  
madd  r3, v1, cb0[1], r3  
madd  r3, v2, cb0[2], r3  
clmp  r3, r3, l(0.0), l(1.0)  
mul   o0, r0, r3  
mul   o1, r1, r3  
mul   o2, r2, r3  
mov   o3, l(1.0)
```

ALU 只能做單一個運算

```
<VEC8_diffuseShader>:  
VEC8_sample vec_r0, vec_v4, t0, vec_s0  
VEC8_mul   vec_r3, vec_v0, cb0[0]  
VEC8_madd  vec_r3, vec_v1, cb0[1], vec_r3  
VEC8_madd  vec_r3, vec_v2, cb0[2], vec_r3  
VEC8_clmp  vec_r3, vec_r3, l(0.0), l(1.0)  
VEC8_mul   vec_o0, vec_r0, vec_r3  
VEC8_mul   vec_o1, vec_r1, vec_r3  
VEC8_mul   vec_o2, vec_r2, vec_r3  
VEC8_mov   o3, l(1.0)
```

很多 ALU 就能做向量運算

(VEC8 是八維向量，一次算八組運算)

高階語言也要為 GPU 專門設計

像是以下的 NVIDIA 的 CUDA 程式，這樣才能充分發揮向量運算的能力

```
cudaArray* cu_array;
texture<float, 2> tex;

// Allocate array
cudaChannelFormatDesc description = cudaCreateChannelDesc<float>();
cudaMallocArray(&cu_array, &description, width, height);

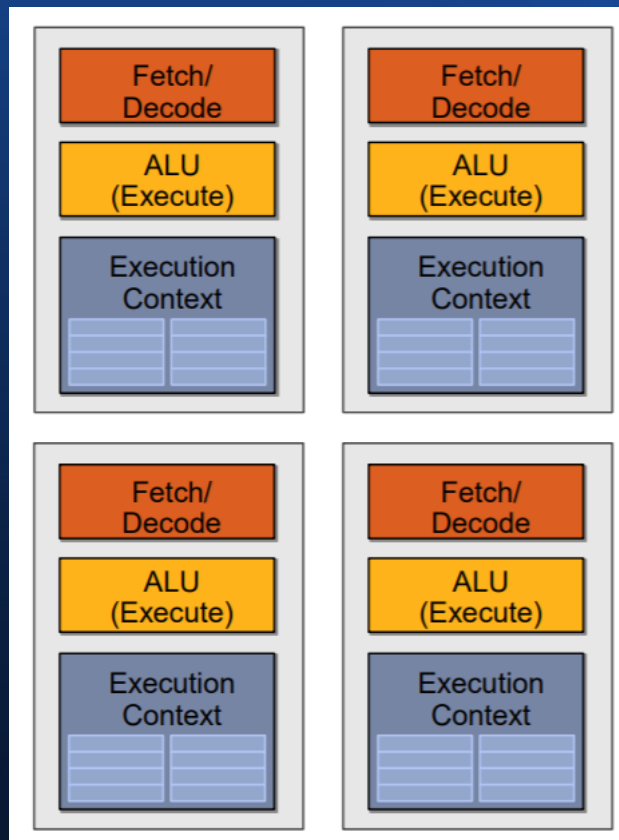
// Copy image data to array
cudaMemcpy(cu_array, image, width*height*sizeof(float), cudaMemcpyHostToDevice);

// Bind the array to the texture
cudaBindTextureToArray(tex, cu_array);

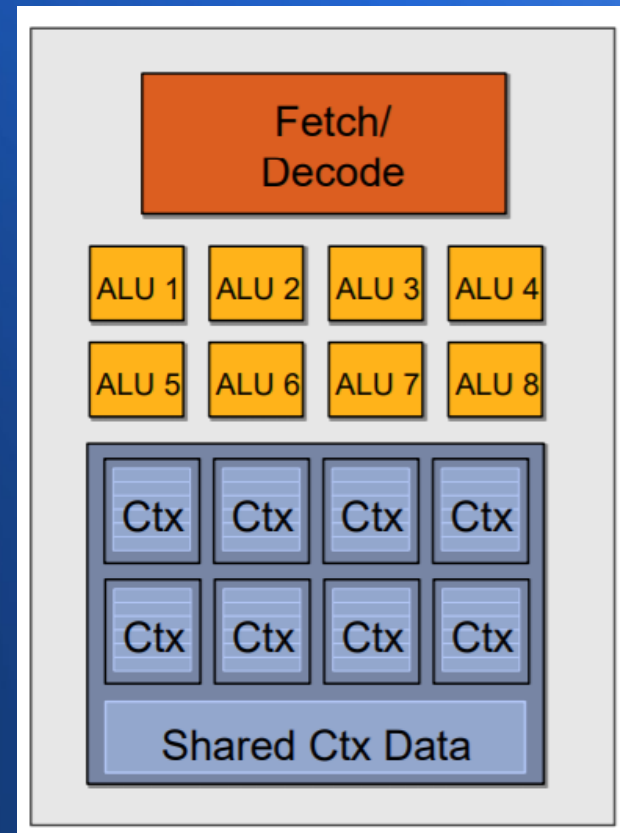
// Run kernel
dim3 blockDim(16, 16, 1);
dim3 gridDim(width / blockDim.x, height / blockDim.y, 1);
kernel<<< gridDim, blockDim, 0 >>>(d_odata, height, width);
cudaUnbindTexture (tex);

__global__ void kernel(float* odata, int height, int width)
{
    unsigned int x = blockIdx.x*blockDim.x + threadIdx.x;
    unsigned int y = blockIdx.y*blockDim.y + threadIdx.y;
    float c = tex2D(tex, x, y);
    odata[y*width+x] = c;
}
```

GPU 和多核心是不一樣的



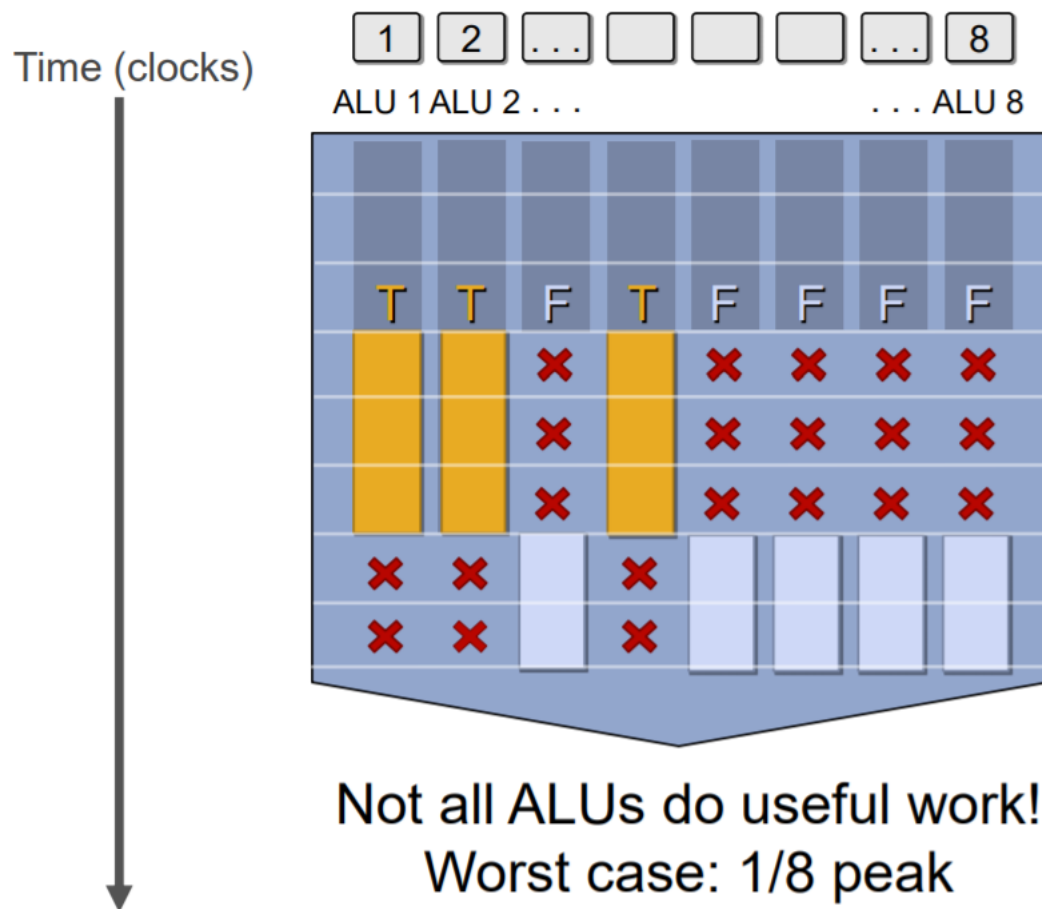
多核心每個 core
通常只有一個 ALU



但是 GPU 則是在一個
core 裡放了很多 ALU

但是 GPU 的平行也會被
條件跳躍所影響，無法全速進行

But what about branches?



<unconditional
shader code>

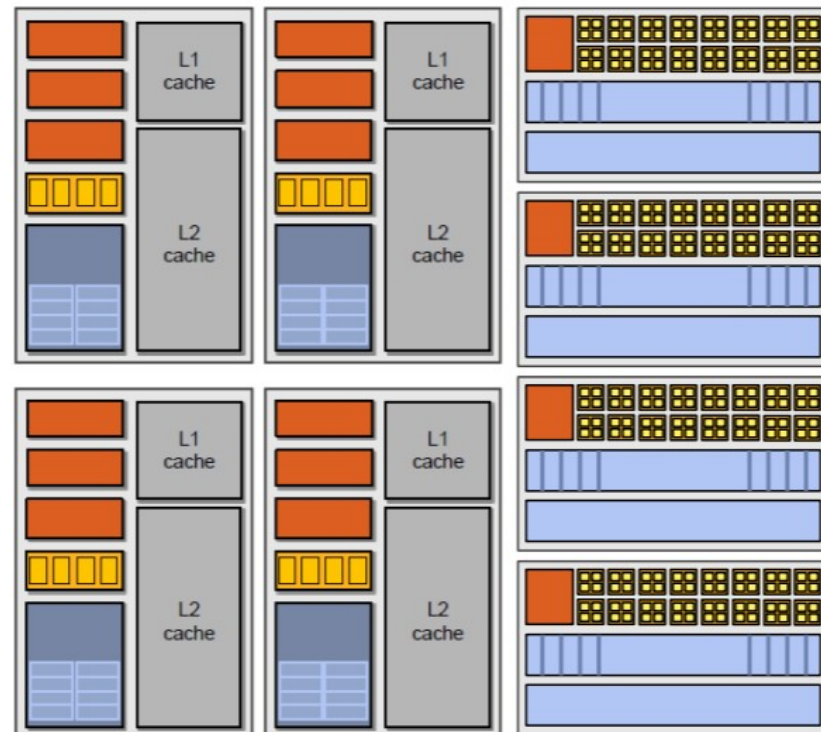
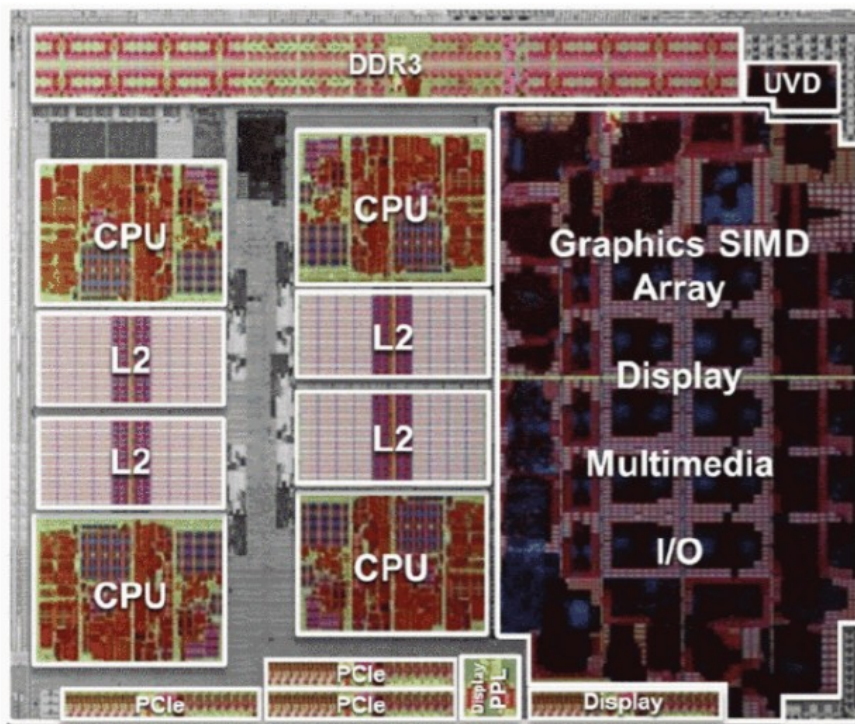
```
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}
```

<resume unconditional
shader code>

現在的電腦架構

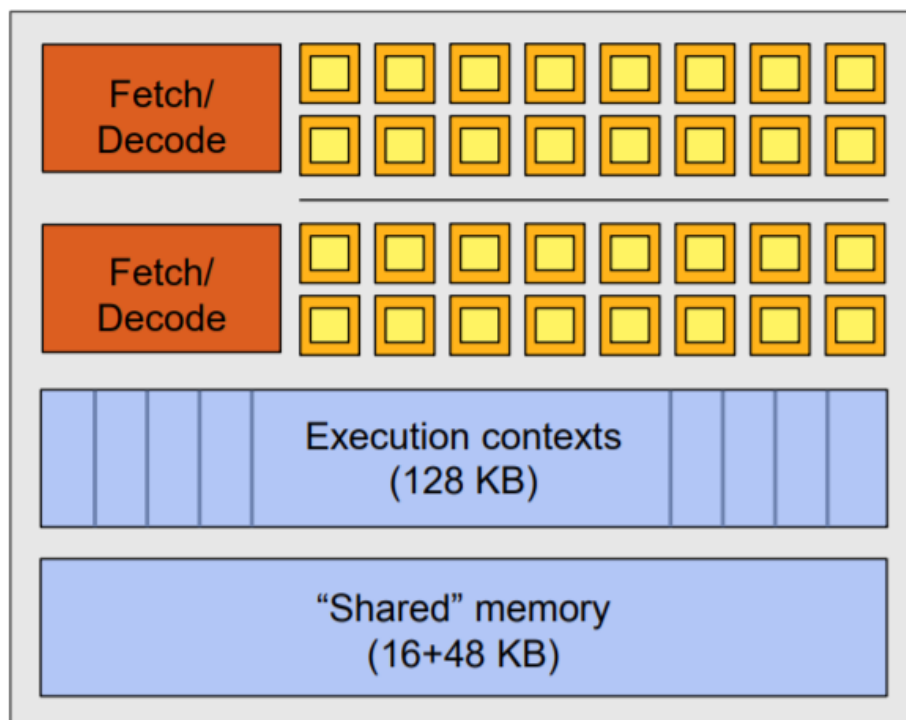
通常是 CPU 和 GPU 攜手合作的局面

AMD A-Series APU (“Llano”)



一個 GPU 的 core 長得像這樣

NVIDIA GeForce GTX 580 “core”

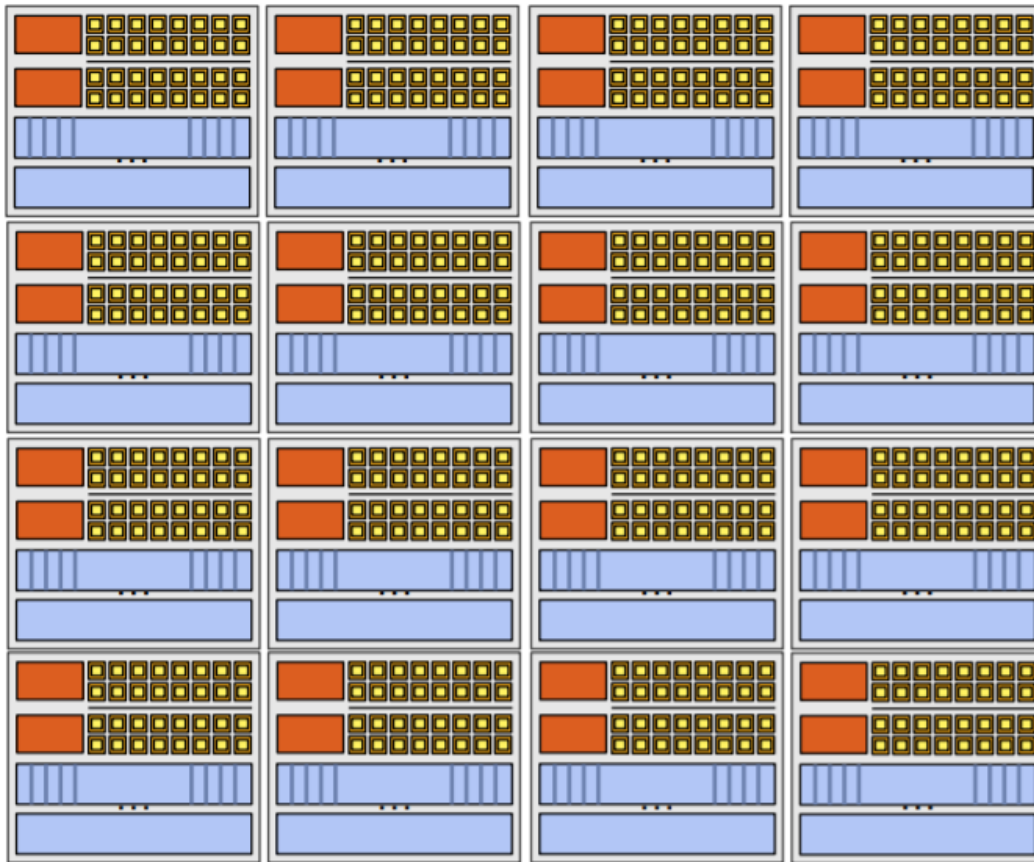


 = SIMD function unit,
control shared across 16 units
(1 MUL-ADD per clock)

- The core contains 32 functional units
- Two groups are selected each clock (decode, fetch, and execute two instruction streams in parallel)

一顆 GPU 裏有很多 core

NVIDIA GeForce GTX 580

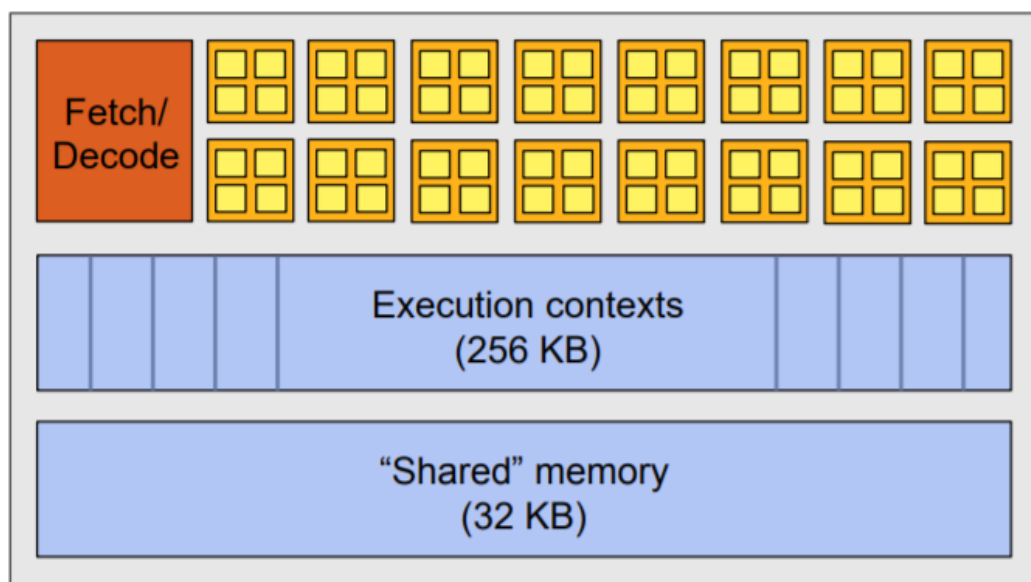


There are 16 of these things on the GTX 480:

That's 24,500 fragments!
Or 24,000 CUDA threads!

不同廠商的 core 長相不同

ATI Radeon HD 6970 “core”

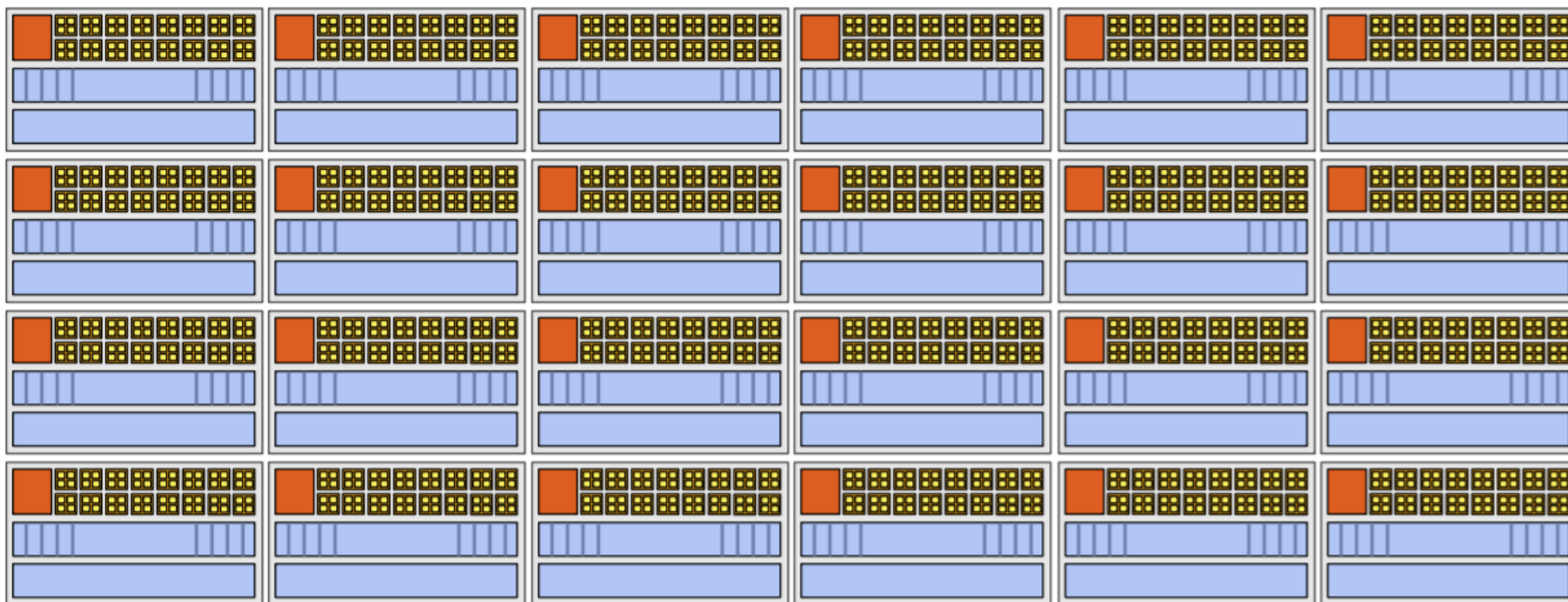


= SIMD function unit,
control shared across 16 units
(Up to 4 MUL-ADDs per clock)

- Groups of 64 [fragments/vertices/etc.] share instruction stream
- Four clocks to execute an instruction for all fragments in a group

而且 core 的數量也有所不同

ATI Radeon HD 6970

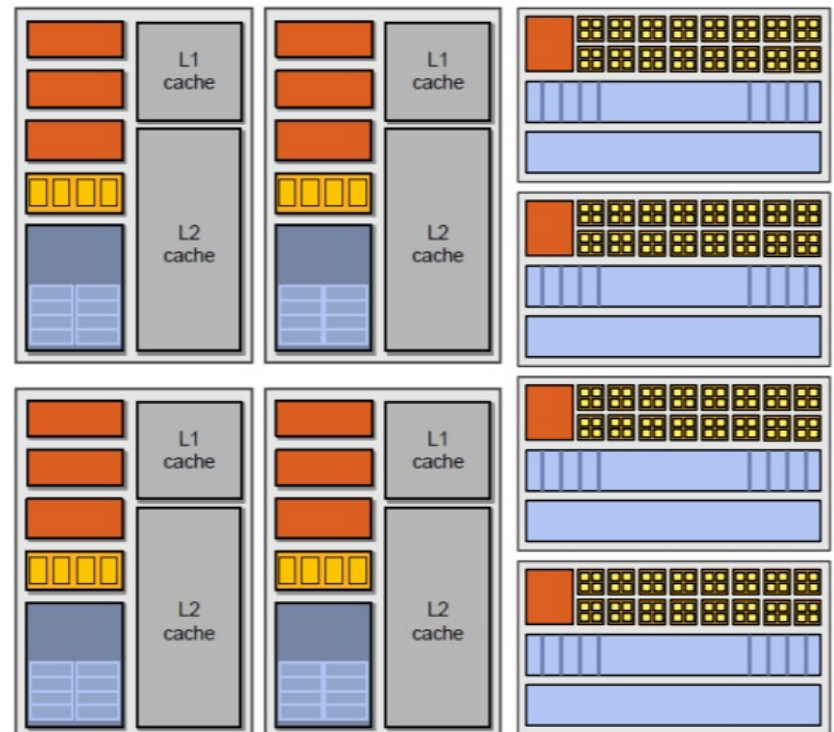
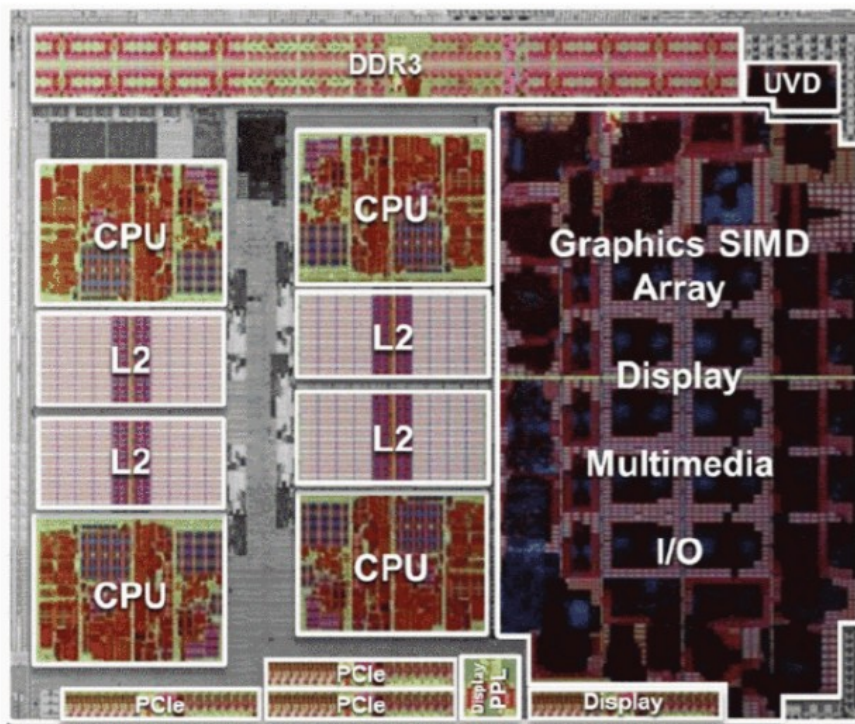


There are 24 of these “cores” on the 6970: that’s about 32,000 fragments!
(there is a global limitation of 496 wavefronts)

融合 CPU 和 GPU 兩者

就可以截長補短，分別做自己擅長的事情

AMD A-Series APU (“Llano”)



這樣

- 您應該看懂 GPU 到底是甚麼了
- 也應該知道現代電腦為何那麼快了...

雖然 CPU + GPU 已經很快了

- 但是沒有最快，只有更快...

對於某些特殊領域

- 那些特製的電腦，常常可以比 CPU + GPU 快上百倍，甚至千倍...

以下讓我們舉兩個例子

這兩個例子是

- 1. Google 的深度學習 TPU
- 2. 比特幣的挖礦機 ...

首先讓我們看看 Google TPU

- TPU 的全名是 Tensor Processing Unit
- 也就是《張量處理器》...

所謂的《張量》

- 差不多就是《高維的向量》
 - 一維是向量
 - 二維是矩陣
 - 三維以上是矩陣的向量
 - 這些高維向量統稱為《張量》

雖然《張量》的數學定義有點複雜

- 但是那些還是留給數學家去研究好了...

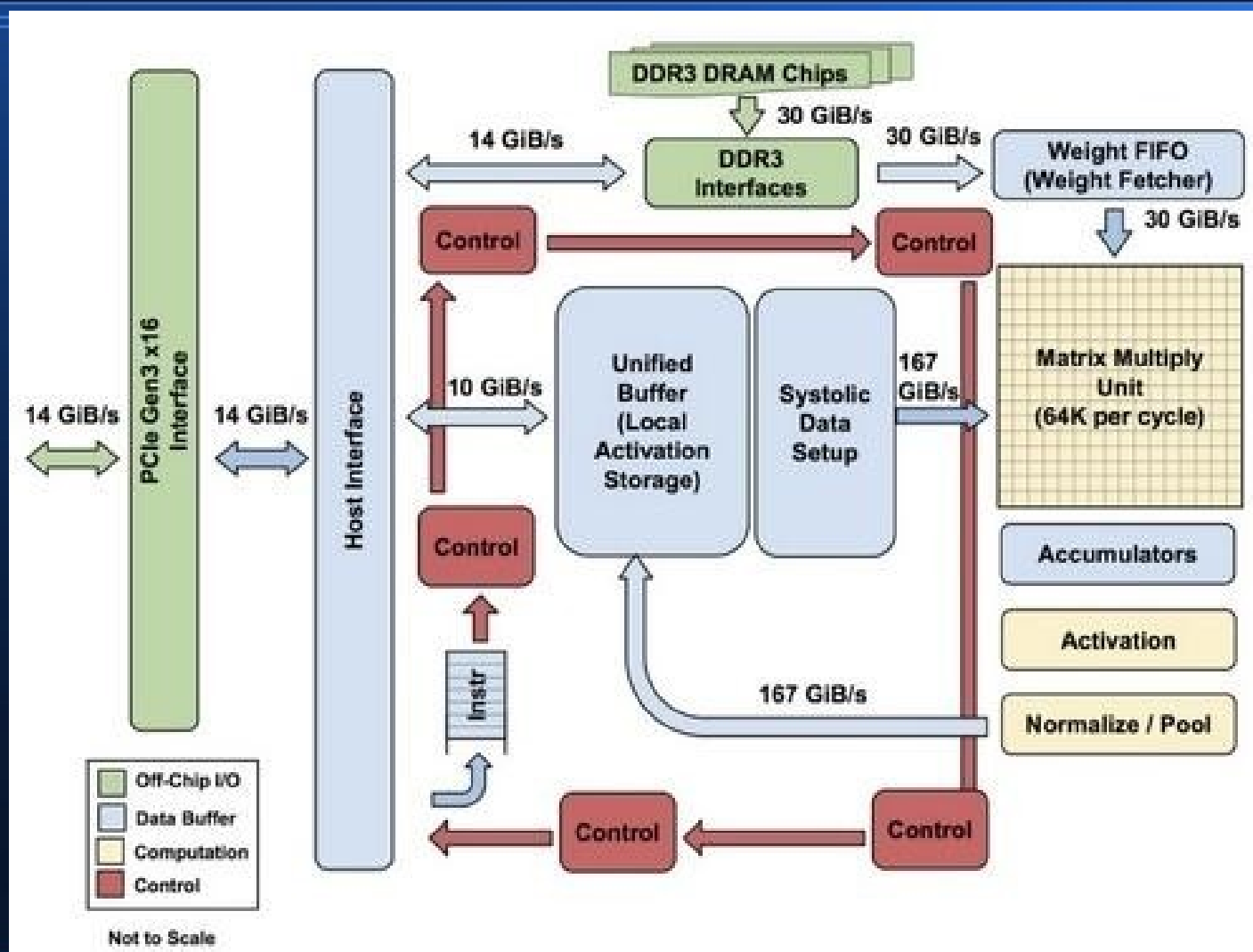
一個 (m, n) 型的張量被定義為一個多重線性映射(multilinear map)

$$T: \underbrace{V^* \times \cdots \times V^*}_{m \text{ copies}} \times \underbrace{V \times \cdots \times V}_{n \text{ copies}} \rightarrow \mathbb{R},$$

其中 V 是向量空間， V^* 是對應的對偶空間。

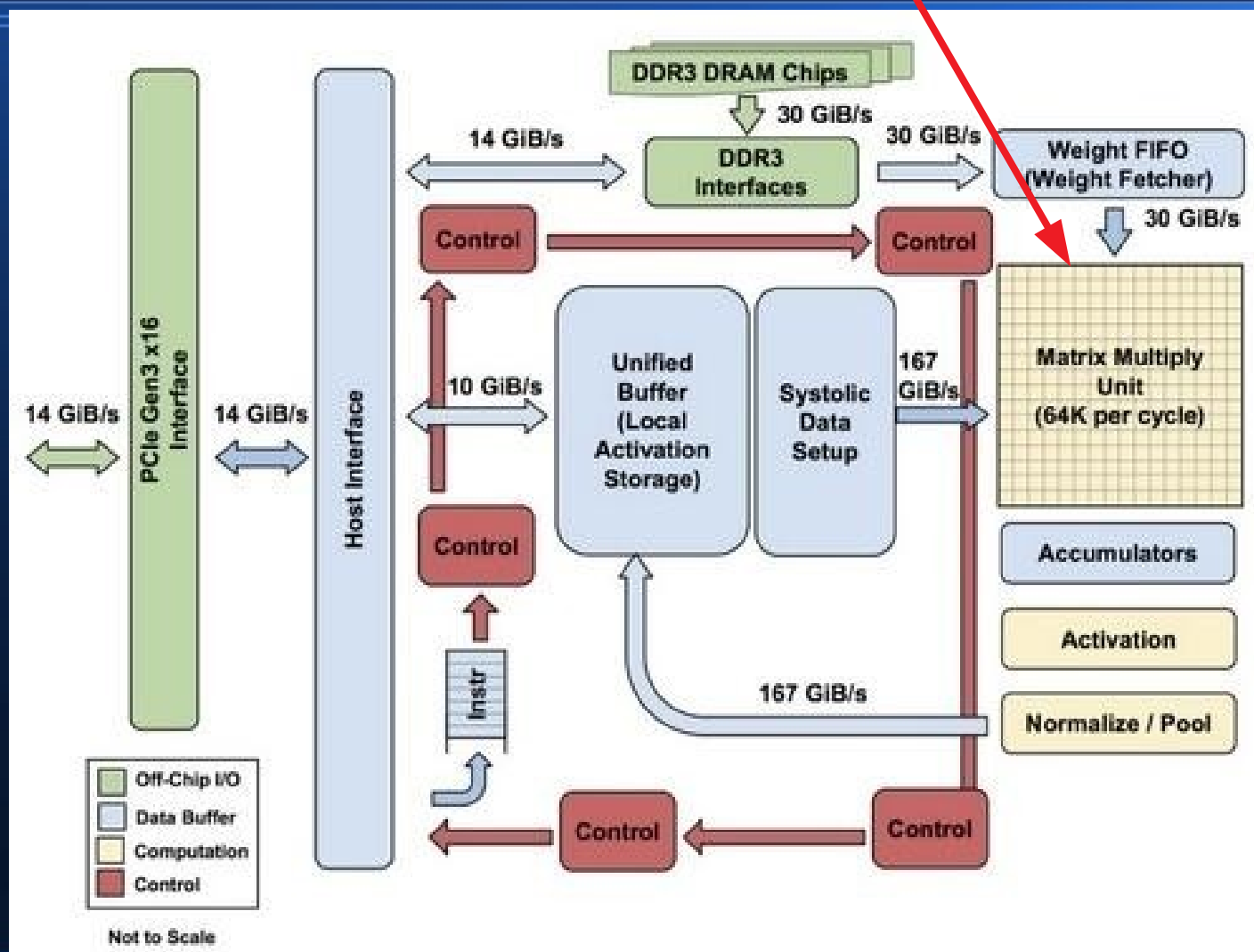
做電腦的先學會怎麼用再說...

TPU 的結構如下



參考：Google 硬體工程師揭密，TPU 為何會比 CPU、GPU 快 30 倍

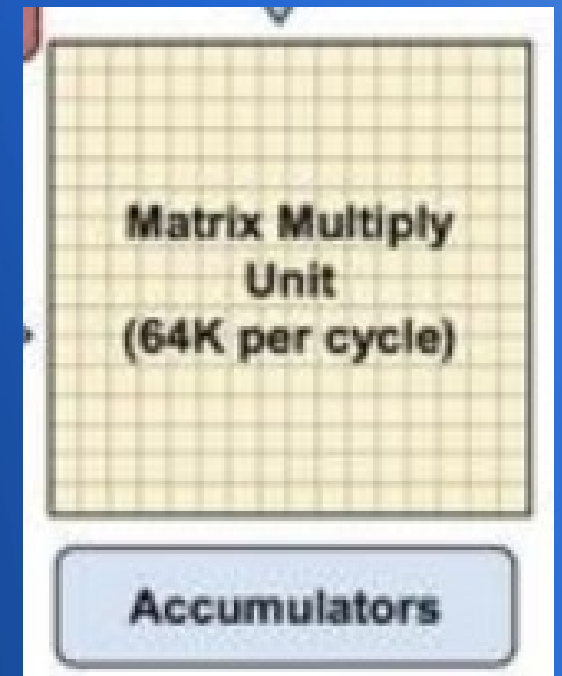
請特別注意 MPU 這一塊



參考：Google 硬體工程師揭密，TPU 為何會比 CPU、GPU 快 30 倍

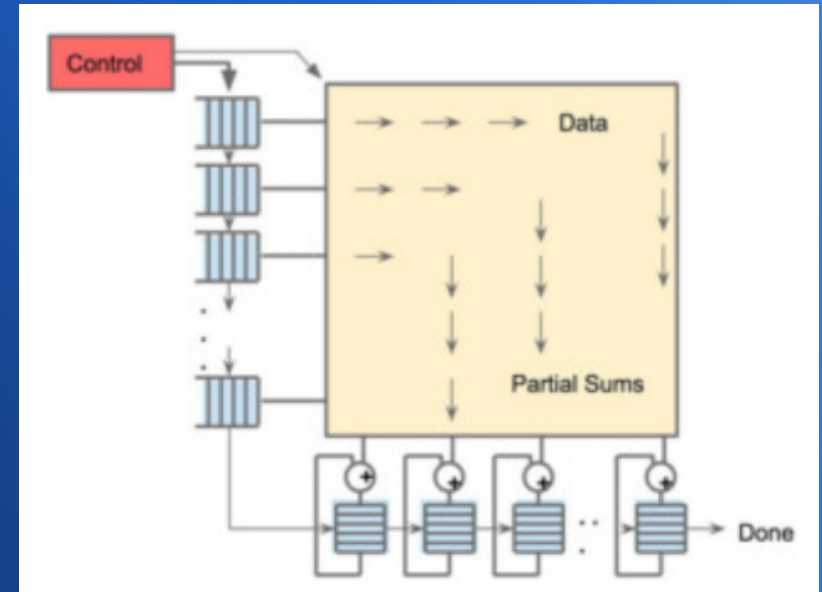
那一整大塊 MPU

- 是 256×256 個《加法與乘法》運算單元
- 而且上面運算完的結果可以直接丟給下面運算



所以

- TPU 一次可以計算一整個矩陣的相乘結果
- 這就比 GPU 一次只能計算《向量相加或相乘》的速度要更快了 ...



這也是為何

- 2016 年 Google 的 AlphaGo 圍棋程式對戰《李世石》的時候，使用了幾千台電腦平行運算才取勝
- 但是到 2017 年 AlphaGo 對戰《柯潔》時，只用了一台擁有 TPU 的電腦就輕鬆贏了...



這樣

- 您應該對 TPU 張量處理器有點概念了...

接著讓我們來看看

- 比特幣挖礦機...

在比特幣剛開始的初期

- 用普通電腦就能挖礦...
- 而且還可以挖到不少...

因為比特幣發明者《中本聰》

- 設下的規定是：
 - 一開始很好挖，鼓勵大家來挖
 - 之後會越來越難挖，價錢才會愈來愈高

果不其然

- 後來比特幣愈來愈難挖了
- 因為《前導零》的長度愈來愈長
- 挖的人愈來愈多
- 而且大家的《軍火》愈來愈強大...

這些軍火一直進化

- 從最早用個人電腦的 CPU
- 後來逐漸改用 GPU（因為 ALU 比較多）
- 然後終於有廠商打造 ASIC 晶片來挖礦 …

比特幣挖礦的關鍵

- 是一種稱為 SHA256 的 hash 簽章函數
- 挖礦者必須不斷的計算 SHA256 的值，去找出符合指定 k 位元前導零的 nonce 。
- 找到 nonce 之後填入並第一個上傳，就會挖到一枚比特幣。

以下是 SHA256 的 hash 範例

SHA256 Hash

Data:

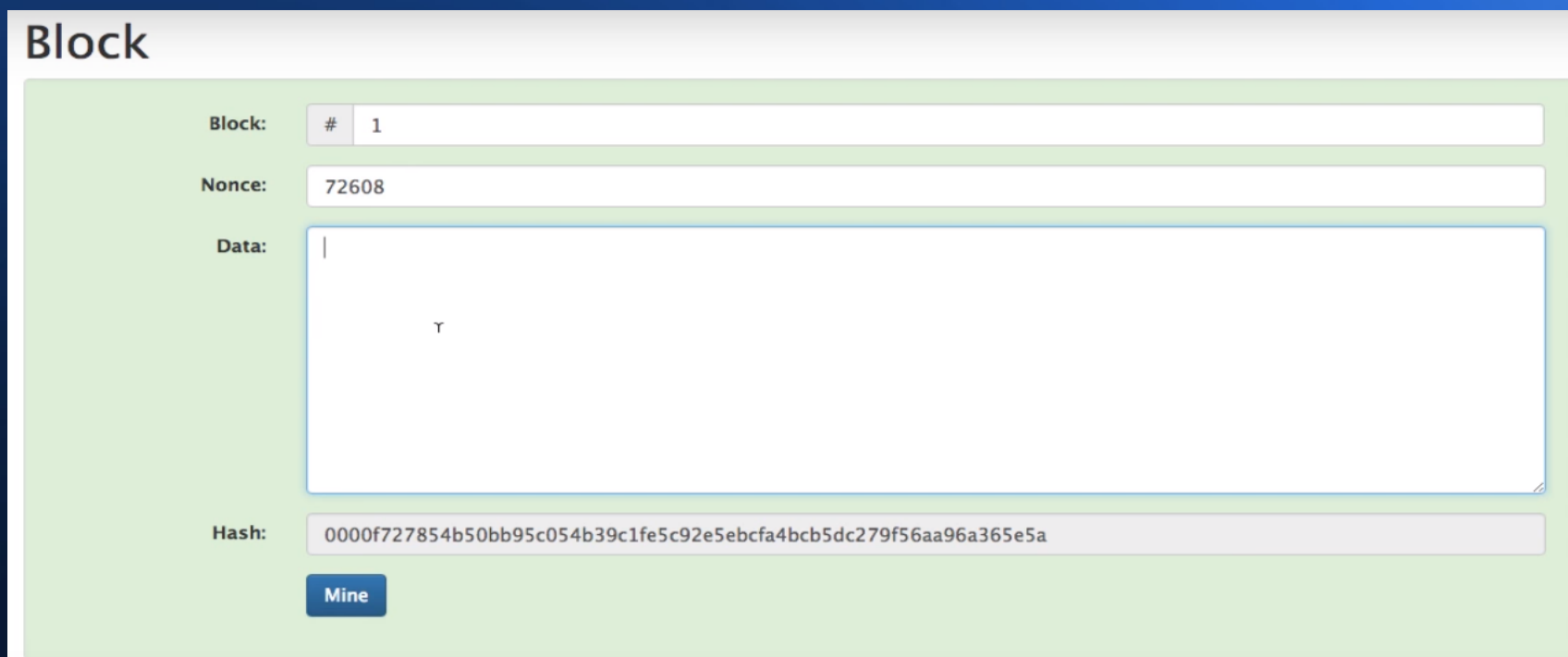
sdc
sdc
e
ev
ev
e
er
erv
e
erv
er

Hash:

795db1eb2c32663c1bd8880001d12fdc6f34a36bb113bbe1d5eba4b271b0ead5

下圖是挖到一枚幣的示意圖

- 圖中的 hash 有四個 16 進位前導零，相當於 16 個二進位前導零（一個 16 進位必須用 4 個二進位表示）



Block

Block: # 1

Nonce: 72608

Data:

Hash: 0000f727854b50bb95c054b39c1fe5c92e5ebcfa4bcb5dc279f56aa96a365e5a

Mine

圖中的 nonce=72608，第一個把簽好章的幣上傳就是挖到了

這些簽章好的證書串成一串

- 就稱為區塊鏈 (Block Chain)

Blockchain

Block: # 1 Y

Nonce: 11316

Data:

Prev: 00000000000000000000000000000000

Hash: 000015783b764259d382017d91a36d206d01

Mine

Block: # 2

Nonce: 35230

Data:

Prev: 000015783b764259d382017d91a36d206d01

Hash: 000012fa9b916eb9078f8d98a7864e697ae83

Mine

Block: # 3

Nonce: 12937

Data:

Prev: 000012fa9b916eb9078f8d98a7864e697ae83

Hash: 0000b9015ce2a08b612161

Mine

所以只要能快速計算 SHA256

- 就可以更有效率的挖礦...

SHA-256 是 SHA-2 演算法的一種

SHA-2，名稱來自於安全雜湊演算法2（英語：Secure Hash Algorithm 2）的縮寫，一種密碼雜湊函式演算法標準，由美國國家安全局研發^[3]，由美國國家標準與技術研究院（NIST）在2001年發布。屬於SHA演算法之一，是SHA-1的後繼者。其下又可再分為六個不同的演算法標準，包括了：SHA-224、SHA-256、SHA-384、SHA-512、SHA-512/224、SHA-512/256。

開發 [編輯]

NIST發布了三個額外的SHA變體，這三個函式都將訊息對應到更長的訊息摘要。以它們的摘要長度（以位元計算）加在原名後面來命名：SHA-256，SHA-384和SHA-512。它們發布於2001年的FIPS PUB 180-2草稿中，隨即通過審查和評論。包含SHA-1的FIPS PUB 180-2，於2002年以官方標準發布。2004年2月，發布了一次FIPS PUB 180-2的變更通知，加入了一個額外的變種SHA-224，這是為了符合雙金鑰3DES所需的金鑰長度而定義^[4]。

SHA-256和SHA-512是很新的雜湊函式，前者以定義一個word為32位元，

SHA-2

概述

設計者 美國國家安全局
首次發布 2001年
系列 (SHA-0), SHA-1, SHA-2, SHA-3
認證 FIPS PUB 180-4, CRYPTREC, NESSIE

細節

編碼長度 224, 256, 384, or 512 bits
結構 Merkle–Damgård construction with Davies–Meyer compression function
重複回數 64 or 80

最佳公開破解

A 2011 attack breaks preimage resistance for 57 out of 80 rounds of SHA-512, and 52 out of 64 rounds for SHA-256.^[1]
Pseudo-collision attack against up to 46 rounds of SHA-256.^[2]

主要算法片段如下

Extend the sixteen 32-bit words into sixty-four 32-bit words:

```
for i from 16 to 63
  s0 := (w[i-15] rightrotate 7) xor (w[i-15] rightrotate 18) xor(w[i-15] rightshift 3)
  s1 := (w[i-2] rightrotate 17) xor (w[i-2] rightrotate 19) xor(w[i-2] rightshift 10)
  w[i] := w[i-16] + s0 + w[i-7] + s1
```

Main Loop:

```
for i from 0 to 63
  s0 := (a rightrotate 2) xor (a rightrotate 13) xor(a rightrotate 22)
  maj := (a and b) xor (a and c) xor(b and c)
  t2 := s0 + maj
  s1 := (e rightrotate 6) xor (e rightrotate 11) xor(e rightrotate 25)
  ch := (e and f) xor ((not e) and g)
  t1 := h + s1 + ch + k[i] + w[i]
  h := g
  g := f
  f := e
  e := d + t1
  d := c
  c := b
  b := a
  a := t1 + t2
```

Produce the final hash value (big-endian):

```
digest = hash = h0 append h1 append h2 append h3 append h4 append h5 append h6 append h7
```

只要把這個演算法 設計成快速硬體電路

- 加入到一般性的電腦架構中（或者做成插件）
- 應該就能製造出挖礦機了.....

因為有了 SHA-256 快速計算電路

- 在挖礦上就可以比一般 CPU 快上千倍....

這樣

- 您應該知道挖礦機
為甚麼會這麼快的原因了...

這就是我們今天的

- 十分鐘看不完系列！

希望您會喜歡！

我們下回見！

Bye Bye!